

Automatic Lipsync/Expressions based on skeleton

Bachelor Thesis



Author: Emmanouil Karydis (Student ID: 4417)

Supervisor: Prof. George Papagiannakis

Co-Supervisor: Dr. Emmanouil Kamarianakis

Technical Co-Supervisor: Marmaras Nick

Department of Computer Science

School of Sciences & Engineering

University of Crete

Heraklion, Greece

October 27, 2023

Abstract

This thesis presents the development of a versatile tool within the Unity3D framework aimed at achieving dynamic lip synchronization and realistic facial expressions on skeleton-based models. The primary objectives of this research include achieving synchronization between user-provided input and corresponding lip movements, creating natural facial expressions through the seamless blending of animations, and designing an interactive user interface (UI) that facilitates input in the form of both text and audio files.

The key methods employed in this endeavor encompass the utilization of Unity3D as the foundational platform, the implementation of C# scripts to control objects and components within the virtual environment, and the integration of the HuggingFace API to transcribe audio input into text. Furthermore, a critical component of the system involves mapping text input to corresponding phonemes using data from the CMU Pronouncing Dictionary. Drawing inspiration from the MPEG4 standards, these phonemes are then transformed into a set of more generic, simplified representations, and animations are generated for each phoneme utilizing Unity's Animation features.

In conclusion, this thesis culminates in the creation of a highly interactive and adaptable environment where users can supply a wide array of input formats, resulting in the production of realistic facial animations. While this research represents a significant step forward in the realm of expressive character animation, there remains ample opportunity for further enhancements and refinements in the pursuit of even greater realism and usability.

Declaration of Authorship

I, Emmanouil Karydis, declare that this thesis titled, "Automatic Lipsync/Expressions based on skeleton", and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signature: Emmanouil Karydis

Date: October 27, 2023

Contents

1	Introduction	1
1.1	Facial Animation Techniques	1
1.1.1	Bone-based animation	1
1.1.2	Rigging	2
1.1.3	Skinning	3
1.1.4	Final Animation and Unity3D	4
1.2	Objectives	5
1.3	Achievements	5
1.3.1	User Interface featuring different Input Formats using Drag and Drop methods	5
1.3.2	Lip Synchronization	5
1.3.3	Expressions	6
1.4	Overview of this work	9
2	State of the Art	9
3	System requirements and tools used	10
4	The Model	11
4.1	Model assets and hierarchy	11
4.2	Model animations	12
4.2.1	Idle Animation	12
4.2.2	Blinking Animation	13
4.2.3	Expression Animations	13
4.2.4	Viseme Animations	21
4.3	Animation Controller	30
4.3.1	Animation Layers	31
5	The User Interface	33
5.1	File Input	36
5.2	Input Layers	36
5.2.1	Phoneme Layer	37
5.2.2	Expression Layer	37
5.2.3	Text Layer	38
5.2.4	Audio Layer	38
5.3	Components Features	38
5.4	Time Layer	40
5.5	Microphone Input	40

6	The Algorithm	41
6.1	User Interface algorithms	42
6.2	Animation algorithms	43
7	Conclusion	44
	References	45
A	Appendix	47
A.1	Bone Hierarchy and Bones used	47
A.2	Controller example	47
A.3	Singleton	49
A.4	Installation Guide	49
A.5	How to install & test this framework	49

List of Figures

1	The mesh of the hand is deformed by the (green) bones. Source .	2
2	Riggers initially utilize a wireframe model (on the left) and proceed to introduce virtual bones (on the right) so Sulley from Monsters, Inc. can be posed. Source	3
3	Skinning a model using 3DS MAX. Source	4
4	A skeleton-based model imported in Unity.	4
5	Text, Expressions, Audio or even individual phonemes can be selected as input.	6
6	The model pronouncing the word "My" in synchronisation with the input.	7
7	The model recreating a happy expression in synchronisation with the input.	8
8	The AI-Powered Lipsync animation from iClone. Source	10
9	The bones on the model used for the animations.	12
10	The blinking animation.	13
11	The default expression.	14
12	The angry expression.	15
13	The disgust expression.	16
14	The fear expression.	17
15	The joy expression.	18
16	The sadness expression.	19
17	The surprise expression.	20
18	The silence viseme.	21
19	The PP viseme.	22
20	The FF viseme.	22
21	The TH viseme.	23
22	The DD viseme.	24
23	The KK viseme.	24
24	The CH viseme.	25
25	The SS viseme.	26
26	The NN viseme.	26
27	The RR viseme.	27
28	The AA viseme.	28
29	The EE viseme.	28
30	The I viseme.	29
31	The O viseme.	29
32	The U viseme.	30

33	An example of an Animator Controller as depicted in the Unity Documentation.	31
34	The Idle Animation Layer.	32
35	The Blinking Animation Layer.	32
36	The Vowel Animation Layer.	33
37	The Consonant Animation Layer.	34
38	The Expression Animation Layer.	35
39	The User Interface, divided in the Input Layer (red box), File Input (green box) and Microphone Input (blue box) sections. . . .	35
40	The File Input section, where the user has chosen the Text File category.	36
41	The Phoneme layer of the UI, annotated by a red box.	37
42	The Expression layer, annotated by a red box.	37
43	The Text layer, annotated by a red box.	38
44	The Audio layer, annotated by a red box, with the corresponding subtitles of the audio files generated (white box).	39
45	The Time layer, annotated by a red box.	41
46	The Time layer, annotated by a red box.	42
47	The Bone hierarchy and the bones used, annotated by a red box. .	47
48	An example of FixedUpdate inside a Controller script.	48
49	A singleton of a public static class is used to get all the component GameObjects we might need. - Part 1.	50
50	A singleton of a public static class is used to get all the component GameObjects we might need. - Part 2.	51

1 Introduction

Speech-driven 3D facial animation has garnered growing interest as a research field, both in academic and industrial settings. This area holds the potential to offer significant advantages across various applications, including virtual reality, film production, gaming, and education. The primary goal of realistic speech-driven 3D facial animation is to autonomously generate lifelike facial expressions for a 3D avatar based on any given speech signal [2]. Speech analysis is time consuming though and absolute real-time synchronisation between speech and generation of a sequence of lip movements and expressions is quite challenging. Other input methods can be used, which can both provide the user with more freedom and achieve real-time lip synchronisation according to the input.

1.1 Facial Animation Techniques

In a survey of facial modeling and animation, Shakir and Al-Azza outlined several categories for facial animation approaches. These categories include blendshape or shape interpolation, parameterizations, facial action coding system-based approaches, moving pictures experts group-4 facial animation, physics-based muscle or skeletal modeling, performance-driven facial animation, and visual speech animation [11]. While numerous studies have aimed to classify and compare various facial animation techniques[11], this particular study concentrates on a bone-based approach, which includes the processes of rigging and skinning.

1.1.1 Bone-based animation

This technique was first introduced in 1988 by Magnenat-Thalmann and colleagues, who presented methods for deformations of the hand's mesh based on joints when gripping an object [6]. Additionally, in 1998, Komatsu demonstrated deformations of the human body driven by an underlying skeleton [4]. This approach aims to simplify the animation process, making it more intuitive. It can be applied not only to animate humans but also any organic figure or mechanical object composed of rigid parts and joints. The initial stage of the skeletal animation process involves positioning a set of points within the mesh, which establishes the skeletal structure of the 3D model. This process is commonly referred to as "rigging," and the end result of the skeletal structure is a hierarchical arrangement of interconnected bones and joints, collectively forming the rig (see Figure 1).

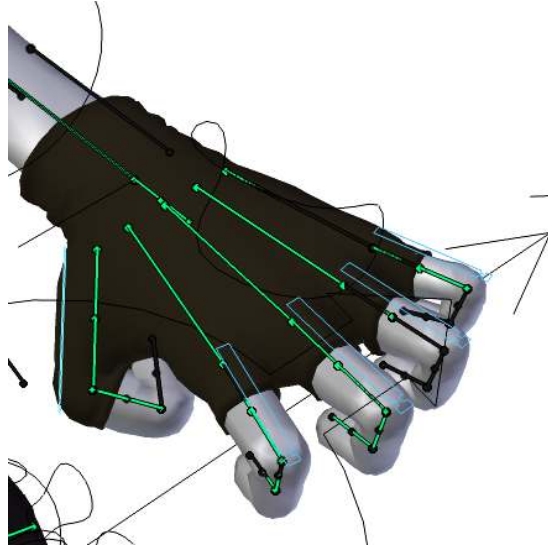


Figure 1: The mesh of the hand is deformed by the (green) bones. Source

1.1.2 Rigging

In the realm of digital character development, artists must comprehend three interconnected systems, each requiring its own knowledge and skills [7]:

1. **Motion System:** This system comprises the character's skeletal structure, which is formed by a combination of joints. Creating an effective motion system demands an understanding of anatomy, range of motion, and kinematics.
2. **Control System:** The control system links controls to the joints, enabling artists to manipulate the motion system. Designing a proficient control system necessitates an understanding of how artists interact with the character model and the ability to devise efficient solutions for complex motion challenges.
3. **Deformation System:** The deformation system defines how the model's geometry will move in relation to the animation of the motion system. Developing a proficient deformation system requires knowledge of anatomy and the ability to preserve the model's form and details during motion.

These three systems collectively constitute what is commonly referred to as "rigging," although sometimes it specifically refers to the motion and control system. Visual effects companies often employ a standardized rigging toolset accessible through a 3D animation program. This toolset must cater to various types of rigs and accommodate different artists' needs while also providing the required output for the production pipeline. Animating a basic 3D object like a cube can be easily achieved through fundamental transformations, such as spatial rotation. However, when we venture into the realm of intricate deformable models like facial animations, the task of controlling and modifying the model becomes far from straightforward. Putting aside the artistic aspects of animation, the primary challenge lies in establishing efficient specifications and motion controls

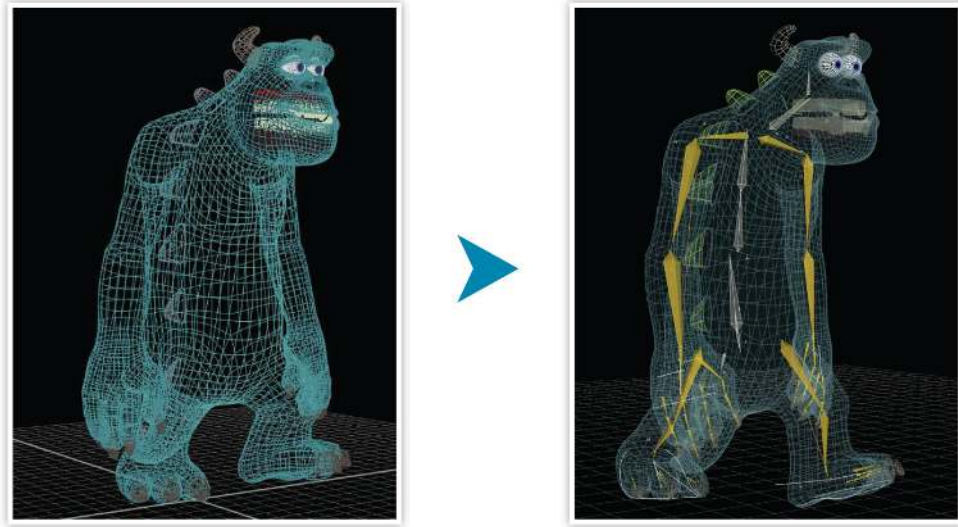


Figure 2: Riggers initially utilize a wireframe model (on the left) and proceed to introduce virtual bones (on the right) so Sulley from *Monsters, Inc.* can be posed. Source

to generate lifelike movements. An example of the rigging process can be seen in Figure 2. While animating a single object defined by a single reference point is a simple task, animating complex models with multiple regions exhibiting various behaviors and conditions proves to be exceedingly challenging. The traditional approach to animating 3D models involves employing a keyframe system to interpolate the positions of all vertices within a specified time frame. Yet, when dealing with increasingly complex models, manipulating each vertex to attain the desired shape becomes impractical. This method demands substantial input from artists, rendering the animation process labor-intensive and time-consuming [9].

1.1.3 Skinning

The skinning technique has been widely used for synthesizing the natural skin deformation of human-like characters in a broad range of computer graphics applications (in Figure 3 a skinning example can be seen by using 3DS MAX). A defined weight for each vertex of the mesh is specified, which indicates how much it can be influenced during a deformation. Each bone has a map that defines the amount of influence it has on each vertex. Numerous skinning techniques have been introduced to enhance the quality of deformation while maintaining real-time computational efficiency. Nevertheless, crafting skinned character models demands extensive manual effort, even from seasoned digital artists equipped with professional software and tools [8].

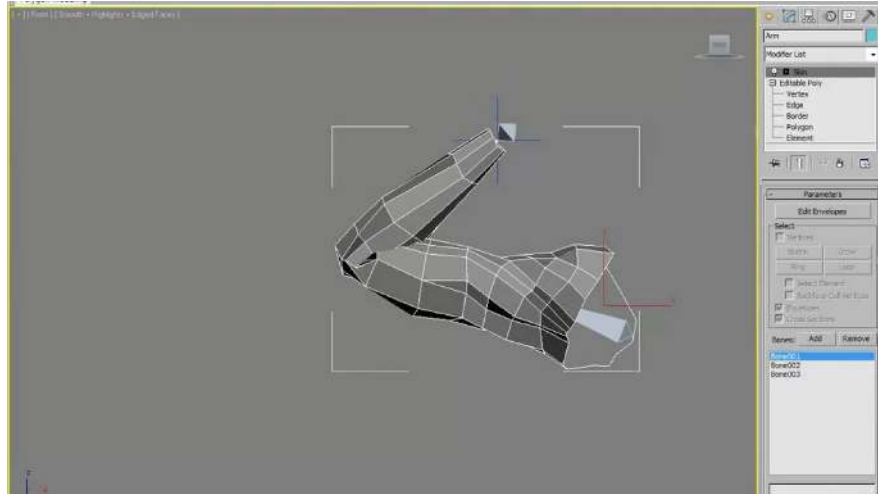


Figure 3: Skinning a model using 3DS MAX. Source

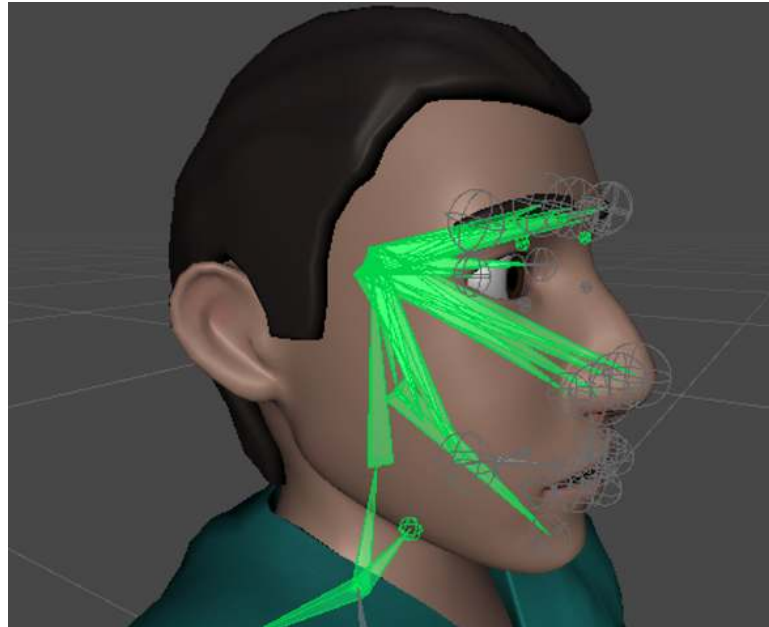


Figure 4: A skeleton-based model imported in Unity.

1.1.4 Final Animation and Unity3D

The final character animation comes to fruition once the skinning process has been finalized and the mesh has been seamlessly integrated into animation software. Game engines such as Unity and Unreal Engine not only provide tools for creating character rigs but also offer robust infrastructure for importing and working with rigged models as can be seen in Figure 4. In this project, we utilized the Unity game engine for the final facial animations. Unity makes it remarkably easy to import rigged models with just a few clicks, allowing you to specify the desired animation type (whether Generic, Humanoid, or Legacy) right from the outset.

Furthermore, Unity supports the importation of materials for rigged characters and provides the capability to create materials within the engine itself. It

boasts compatibility with various 3D file formats, including .fbx, .obj, .dae, .3ds, and .blend. Moreover, the model itself is provided, a skeleton-based model which is used to develop the facial animation and lip synchronization process.

1.2 Objectives

As mentioned above, by using an already built skeleton-based model, the goal is to build a tool that handles a variety of different input formats and produce realistic facial animation on the model, according to the input given. While there are existing tools serving a similar purpose, the distinctive advantage of my approach lies in the inclusion of a user-friendly and practical User Interface. Additionally, it offers simultaneous support for both Expressions and Lip Synchronization. The approach also takes into account humanoid facial behaviors, such as idling and blinking, to attain a more realistic and comprehensive result.

1.3 Achievements

1.3.1 User Interface featuring different Input Formats using Drag and Drop methods

The user interface that is too complex actually makes users have difficulty in using, and of course, errors can occur in use. According to research conducted by User Interface Engineering, Inc., it is known that 60% of the time is wasted because people cannot find the information they want to get and this impact on decreasing productivity and increasing frustration [10]. The assessment of the successful implementation of a system is conducted to gauge the user's proficiency in utilizing the system, with the aim of ensuring that the system's objectives are met. Users have the flexibility to select from a range of diverse input formats, including default options provided by the tool like various Expressions (such as Anger, Fear, etc.) and Phonemes (AA, PP, Sil, etc.). Additionally, users can introduce their own input in the form of text or audio files, as viewed in Figure 5. These input elements can be combined simultaneously to generate the intended facial animation. This underscores the importance of implementing a user-friendly interface, ensuring that users can effortlessly manage the tool and select the desired input components for the model's output process.

1.3.2 Lip Synchronization

Lip syncing is an integral part of an animation that makes the character come alive. In the simplest terms, lip-syncing can be termed as the process and way in which the sound is synchronized to the characters in animation. It is lip-syncing

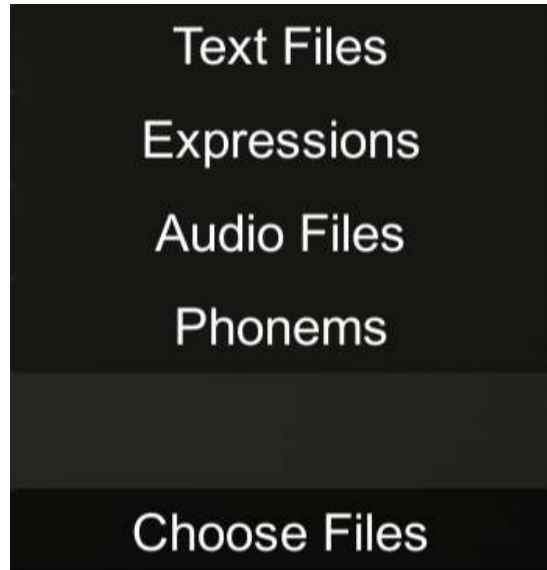


Figure 5: Text, Expressions, Audio or even individual phonemes can be selected as input.

which brings life to the characters and makes them look talking. Lip synchronization is employed in various application areas such as animated movies, computer games, and virtual reality (VR). Max Fleischer's 1926 short film, "My Old Kentucky Home," represents one of the earliest attempts at lip sync in animation. This technique remains prevalent today, as it is utilized in animated movies and TV shows like "Shrek," "Lilo & Stitch," and "The Simpsons" to make their animated characters convincingly talk. Phonemes represent the smallest sound units that compose words and sentences. For instance, the word "cat" contains three phonemes: /k/, /a/, and /t/. Each phoneme is associated with a specific mouth shape or position that aids in producing the sound. A viseme serves as the visual representation of a phoneme within a spoken language, specifying the facial and mouth configuration during word pronunciation. Through the lip sync feature, developers can extract the viseme sequence along with its duration from generated speech, facilitating synchronization with facial expressions.

This thesis successfully achieved synchronized, lifelike lip movements in response to user input using phonemes as the key component of the input as can be seen in Figure 6, resulting in an attractive outcome where the skeleton-based model imitates the words provided by the user through the user interface (UI).

1.3.3 Expressions

Previous research affirms that facial behaviors hold greater significance in terms of achieving human-like qualities and eliciting empathy than merely focusing on visual aesthetics [5]. Several experiments have demonstrated the accuracy

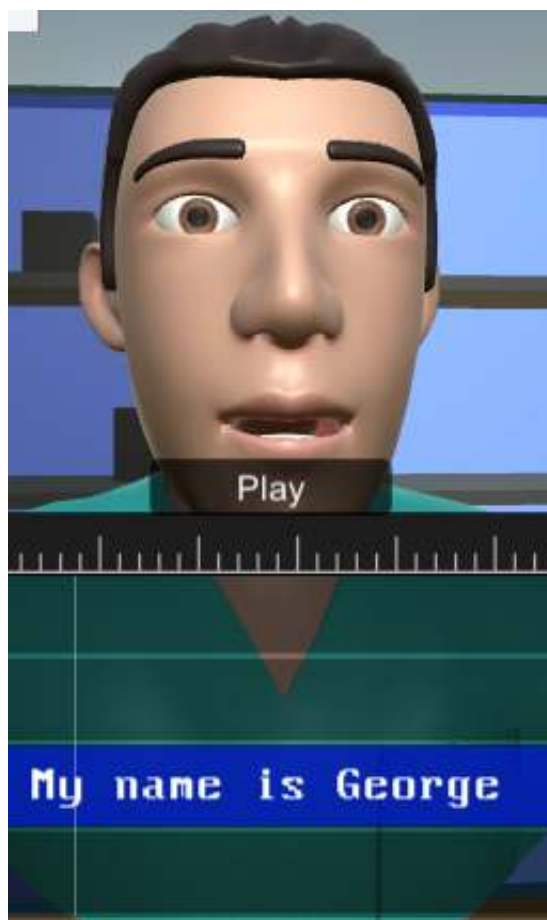


Figure 6: The model pronouncing the word "My" in synchronisation with the input.

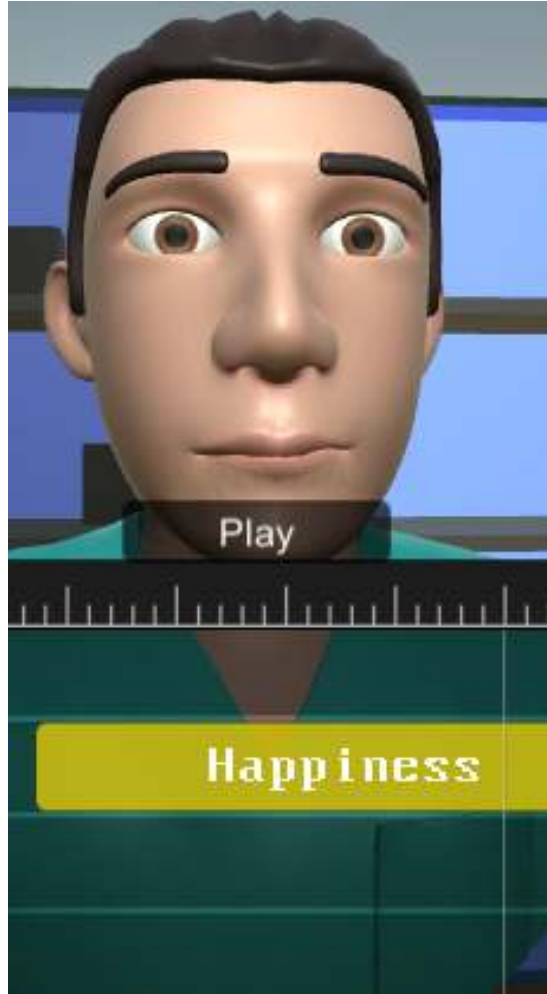


Figure 7: The model recreating a happy expression in synchronisation with the input.

of judging facial expressions in different contexts, such as psychiatric patients' emotional states and responses to stress interviews or unpleasant stimuli. Additionally, observers were able to discern whether individuals were exposed to pleasant or unpleasant content based on their facial expressions [1]. This body of evidence supports the ability of individuals to perceive and interpret basic emotional cues conveyed through facial expressions. In 1975, Ekman and Friesen established the framework for the six primary facial expressions: anger, disgust, fear, joy, sadness, and surprise. It is believed that all other emotions can be conveyed through combinations of these fundamental expressions. These six basic facial expressions were incorporated into this project, and when combined with the lip synchronization feature, they contribute significantly to the realism of the model's facial animation (see Figure 7).

1.4 Overview of this work

The rest of this work is structured as follows:

- In section 2 we dive into the state of the art of Lip synchronisation and its implementations on different industries.
- Section 3 contains a small description of the system requirements and the tools that are necessary for the implementation and the execution of this project.
- The structure of the model used in this thesis and the description of the animations for the facial expressions are provided in section 4.
- In section 5 we describe the User Interface of the project, as well as its features.
- Lastly, in section 6 the algorithms themselves are explored in more depth.

At first, the project was mostly focused on real-time facial animation using microphone input, but later it evolved into something different, as it was difficult to produce a real-time outcome using speech recognition software and APIs. The route that the project took was focused on other input formats such as Keybinds in the beginning, which was mostly done for testing the facial animations, and later through files. In the end the need for a User Interface was clear, as it made it easier for the user to interact with the tool and achieve the expected outcome.

2 State of the Art

Nowadays there are a lot of tools and APIs that can achieve realistic lip sync implementations on skeleton-based models, one of which can be seen in Figure 8. The majority of them are audio driven, for example they take input from the microphone and process the information of the audio sample either through a speech recognition or an audio pattern recognition software. In cases of pattern recognition, the software tries to make a prediction based on the input and after a lot of training, it manages to produce a realistic result. Also motion capture is commonly used in both film and gaming industry to achieve lip synchronisation. It requires a lot of high quality cameras and a trained tracking model. As the real life actor moves, talks and make different facial expressions, the tracking model gathers data of the facial movements, which are later used to make the corresponding 3D animations. These techniques, although they might be very effective in achieving realism are quite expensive in time and resources and are



Figure 8: The AI-Powered Lipsync animation from iClone. Source

mostly used with blendshape facial animation techniques. In this thesis we will discuss about the speech recognition tools used for lip sync and the methods used to generate facial animations through text. Some programming languages have speech recognition libraries (for example C# has the `SpeechRecognitionEngine Class`). Google Cloud Platform also has a Speech-to-text tool, as well as OpenAI which can transcript audio input to text output. After the initial conversion, the produced text is analysed through a Dictionary which segments every word to its corresponding series of phonemes. Phonemes serve as a valuable tool for animators, enabling them to craft lifelike and uniform lip synchronization by offering a standardized set of mouth shapes adaptable to various characters and languages. They simplify the animation workflow by minimizing the requirement for extensive keyframes and transitions, resulting in smoother and more authentic lip movements.

3 System requirements and tools used

First of all, the project is a tool built for Unity3D, which is why it is crucial for the user to have it installed already. It uses some of Unity's animation features such as Blend Trees and Animators, which are essential for the smooth transitions between animations and for simultaneous working animations. Unity works with Microsoft Visual Studio to establish operational scripts. These scripts have been created using the Visual Studio Integrated Development Environment and are written in the C# programming language. Additionally, a skeleton-based model containing the requisite facial bones (further analysis of which will be conducted in another section of this thesis) is essential for the system to operate effectively. It's worth noting that the model doesn't need to encompass the entire body; only the

facial bones on the models are utilized in this project. The system accepts input from pre-existing components (such as Expressions and Phonemes) and can also process ASCII text from a .txt file or audio from a .wav file. The text file should be written in English for the tool to work properly, although the system will try to generate animations from each letter individually, if a word in the text file is not recognised. These external files should be located in a designated folder within the project structure to enable the tool to access their content (specifically, in "Assets/Resources/DragAndDrop"). Furthermore, an experimental feature has been integrated into the system, allowing it to interpret microphone input and convert it into facial animations. Hence, the user will need access to a microphone device for this functionality.

4 The Model

4.1 Model assets and hierarchy

The model in this project is a skeleton-based model with some materials and textures already built which make it look realistic, so that the lip movements can be seen more clearly. Its skeleton hierarchy consists of the typical skeleton-based bone hierarchy. It does not contain arm and leg bones, however it consists of a variety of different facial bones, which are essential to the realistic animations that are made for the lip synchronisation project. Most of the bones have a base, which then contains one or more subsections of the same bone, something that can provide more freedom of movement on the model's face. Even though the model used provides a great variety of different bones, the ones used for the facial animations and expressions are the following:

- 8 Lip bones (red)
- 1 Tongue bone (green)
- 1 Jaw bone (yellow)
- 4 Eye lid bones (blue)
- 4 Eye brow bones (purple)

These bones are depicted in Figure 9 via the color written in parenthesis.

Within the MPEG4 specification, there existed a total of 16 primary Facial Points (FPs) that could be influenced by Facial Animation Parameters (FAPs), and an additional 2 secondary points specifically for the mouth. Nevertheless, if the system were to manage a multitude of facial points concentrated solely on the lips, it would undoubtedly introduce complexity. Additionally, the task of locating or creating a model with an extensive array of bones for the foundation of animations would become increasingly challenging. This is why, in the context

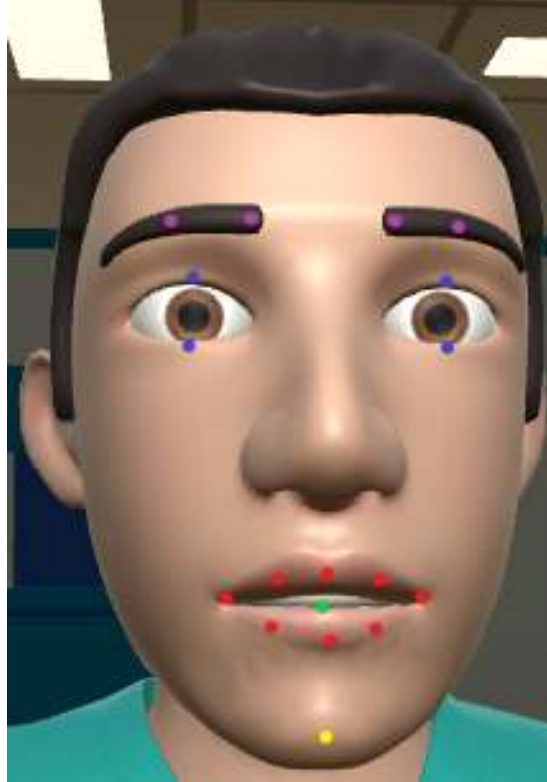


Figure 9: The bones on the model used for the animations.

of this thesis, a simplification was adopted, limiting the number of lip bones to just 8. With these 8 lip bones, a wide range of distinct facial animations could be constructed. To maintain simplicity, the model was designed to feature only a single tongue and a solitary jaw bone. These elements prove valuable when the model needs to articulate phonemes that entail mouth opening or closing, or extending the tongue. Lastly, the utilization of fundamental eyelid and eye-brow bones sufficed to achieve the realistic eye movements essential for the facial expressions recreated by the model.

4.2 Model animations

Numerous diverse animations have been integrated into the model, encompassing a wide spectrum of functions. While some of these animations pertain to the precise movements of the lips during speech, others target various facets of facial animation, contributing to an enhanced level of realism in both appearance and behavior of the model.

4.2.1 Idle Animation

In the field of human motion research, the majority of studies tend to focus on the portrayal of emotions, often overlooking the notion of idle movements [3].



Figure 10: The blinking animation.

In reality, people engage in various movements even when they are not actively engaged in a specific task. However, avatars and robots typically remain static in a standby mode, lacking the natural, continuous movement that characterizes real-life human behavior. As a result, a simple idle animation has been implemented on the model, which mimics a breathing effect.

4.2.2 Blinking Animation

The act of blinking represents one of the most significant cues in the process of forming impressions of a person, in this case of an avatar. An animated blink rate of 18 blinks per minute, when employed with a human-style avatar, generates the most amicable and friendly impression. On the contrary, elevated blink rates, such as 36 blinks per minute, tend to convey a sense of passivity, while lower blink rates, for example, 9 blinks per minute, tend to impart an impression of intelligence. These findings provide valuable guidelines for adjusting the avatar's blinking rate as a means to control its attractiveness. Nevertheless, even though the blinking rate of an avatar's animation holds as much importance as factors like eye gaze, it has not received thorough scrutiny in the context of avatar representation[12]. In this project, a blinking animation for the model is created (see Figure 10), which is activated in a random manner but in average once every 4 seconds, which means 15 blinks per minute in average.

4.2.3 Expression Animations

As it was mentioned before, all emotions can be expressed through combinations of six basic expressions. An animation has been created for each one of them taking inspiration from the MPEG4 specification about Expressions. Moreover,

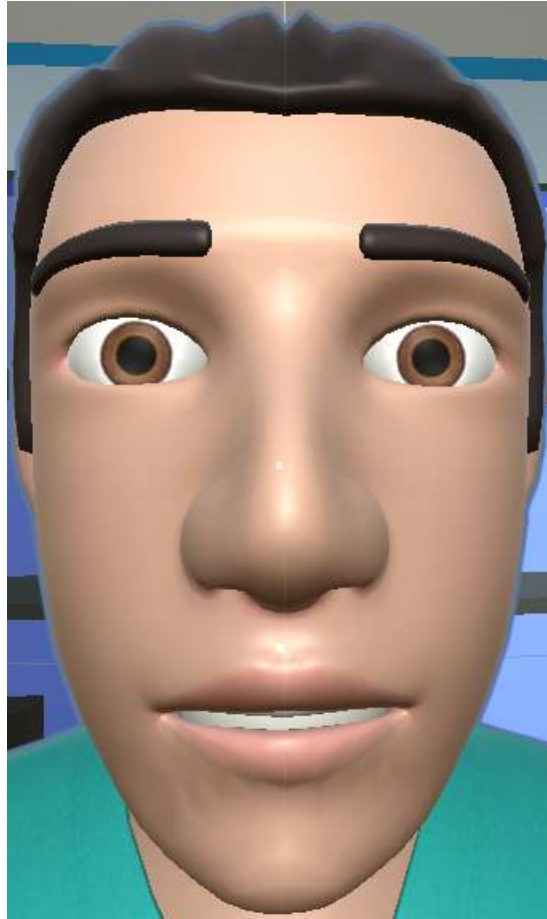


Figure 11: The default expression.

in this project another expression animation has been added, the default one, as in some cases the model might show no particular emotion.

- **Default**

Description:

The default animation is the state of the model where it shows no emotion, the facial expression is exactly the same as the one that the original model has (see Figure 11).

- **Anger**

Description:

- The inner eyebrows are pulled downward and together.
- The eyes are wide open.

The animation can be seen in Figure 12

- **Disgust**

Description:

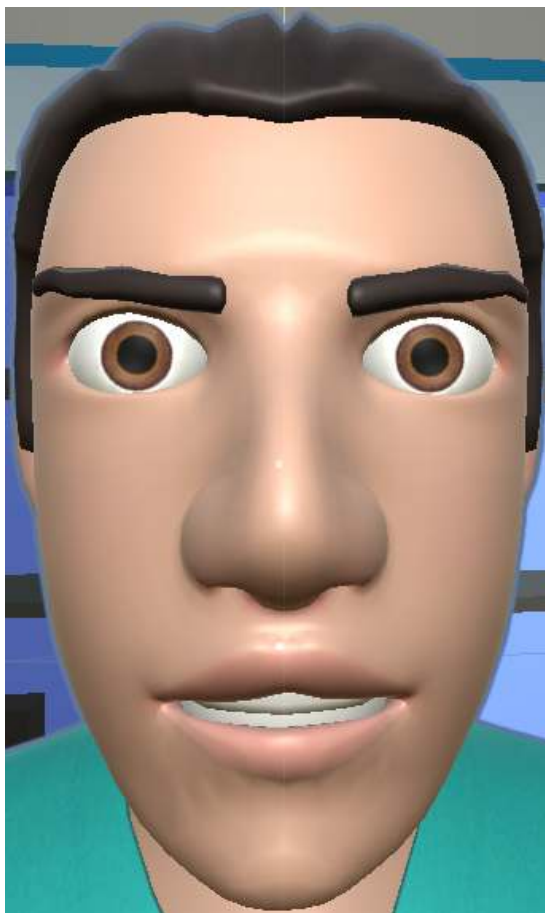


Figure 12: The angry expression.



Figure 13: The disgust expression.

- The inner eyebrows are pulled downward and together.
- The eyes are wide open.
- The lips are slightly opened to expose the teeth

The animation can be seen in Figure 13

- **Fear**

Description:

- The eyebrows are raised and pulled together.
- The inner eyebrows are bent upward

The animation can be seen in Figure 14

- **Joy**

Description:

- The eyebrows are relaxed
- The mouth is open and the mouth corners a little pulled back toward the ears.

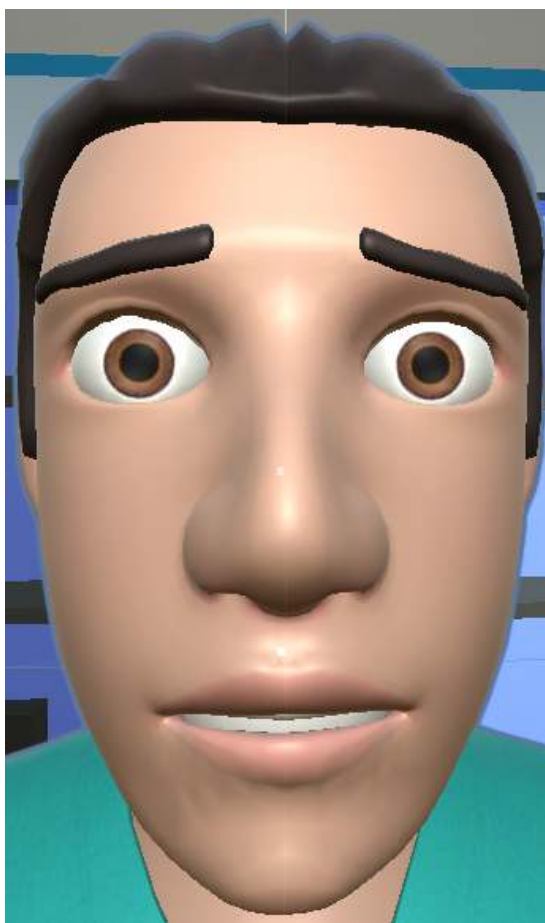


Figure 14: The fear expression.

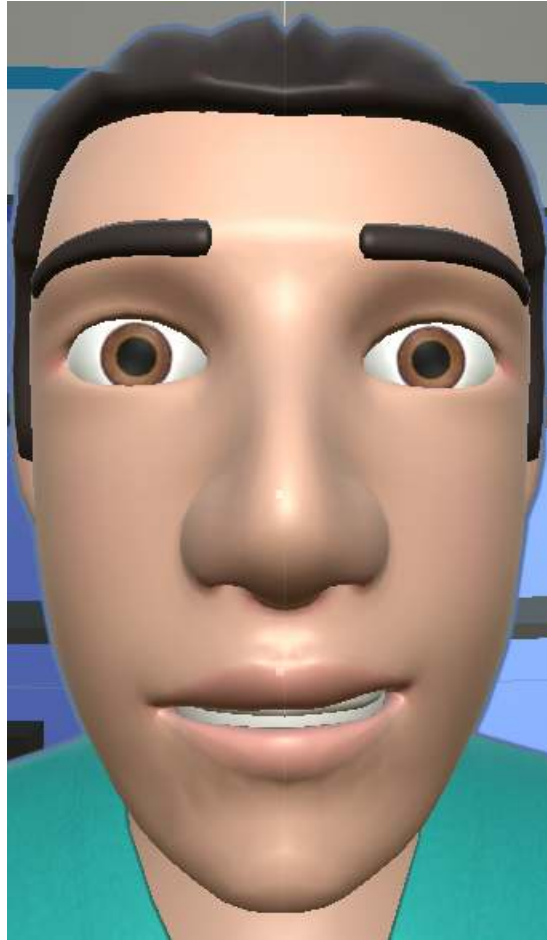


Figure 15: The joy expression.

The animation can be seen in Figure 15

- **Sadness**

Description:

- - The inner eyebrows are bent upwards
- The eyes are slightly closed

The animation can be seen in Figure 16

- **Surprise**

Description:

- The eyebrows are raised.
- The upper eyelids are wide open.
- The lower eyelids are relaxed

The animation can be seen in Figure 17

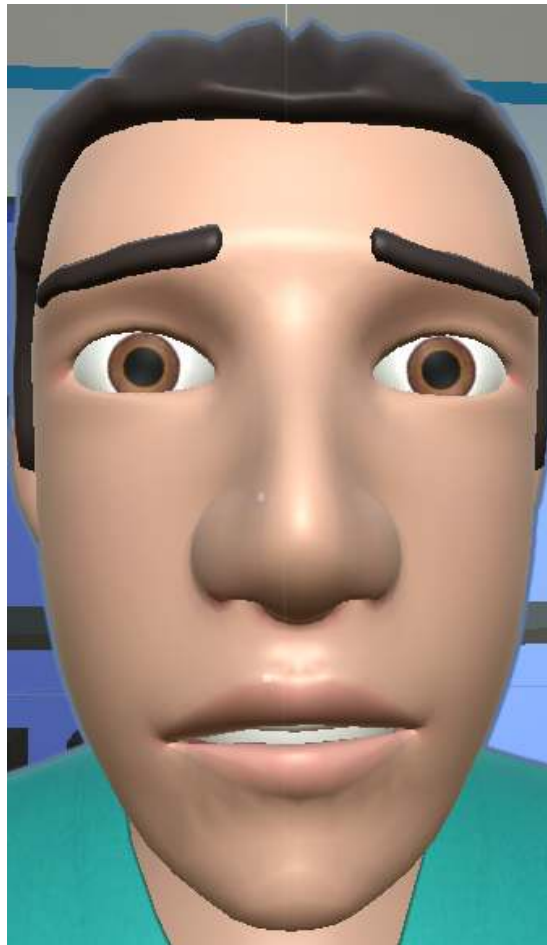


Figure 16: The sadness expression.

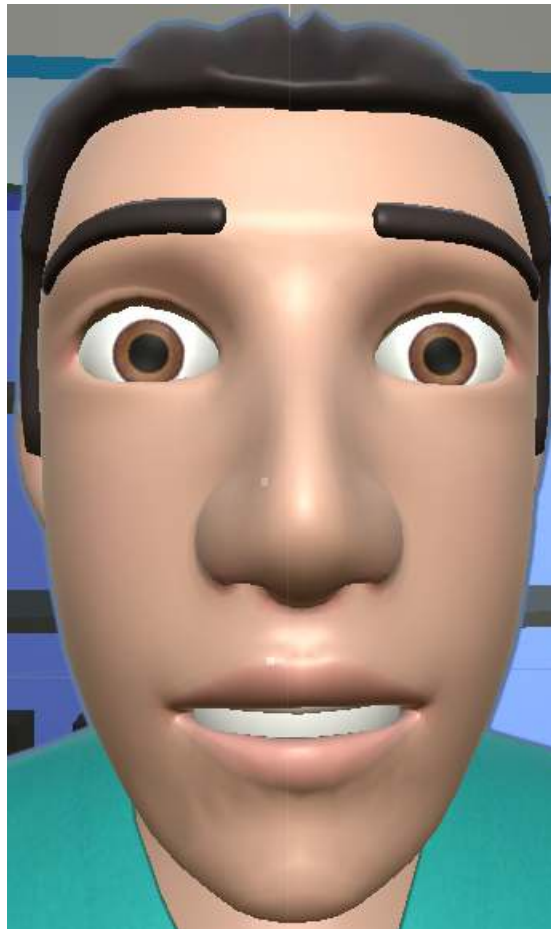


Figure 17: The surprise expression.

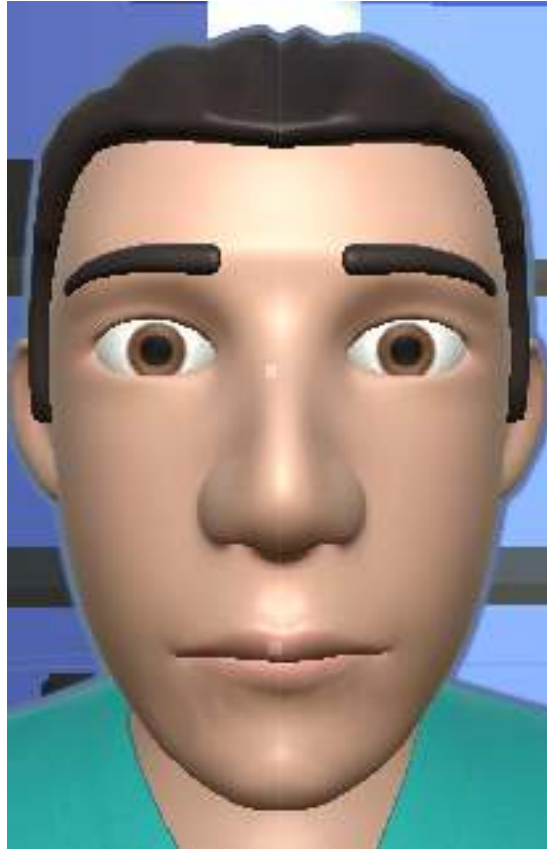


Figure 18: The silence viseme.

4.2.4 Viseme Animations

An animation for each different viseme has been made, according to the MPEG-4 viseme specification.

- **Sil**

Corresponding phonemes: Neutral

Example: (none-silence)

The animation can be seen in Figure 18

- **PP**

Corresponding phonemes: p, b, m

Example: Put, bat, mat

The animation can be seen in Figure 19

- **FF**

Corresponding phonemes: f, v

Example: fat, vat

The animation can be seen in Figure 20



Figure 19: The PP viseme.



Figure 20: The FF viseme.



Figure 21: The TH viseme.

- **TH**

Corresponding phonemes: th

Example: Think, that

The animation can be seen in Figure 21

- **DD**

Corresponding phonemes: t, d

Example: tip, doll

The animation can be seen in Figure 22

- **KK**

Corresponding phonemes: k, g

Example: call, gas

The animation can be seen in Figure 23

- **CH**

Corresponding phonemes: tS, dZ, S

Example: chair, join, she

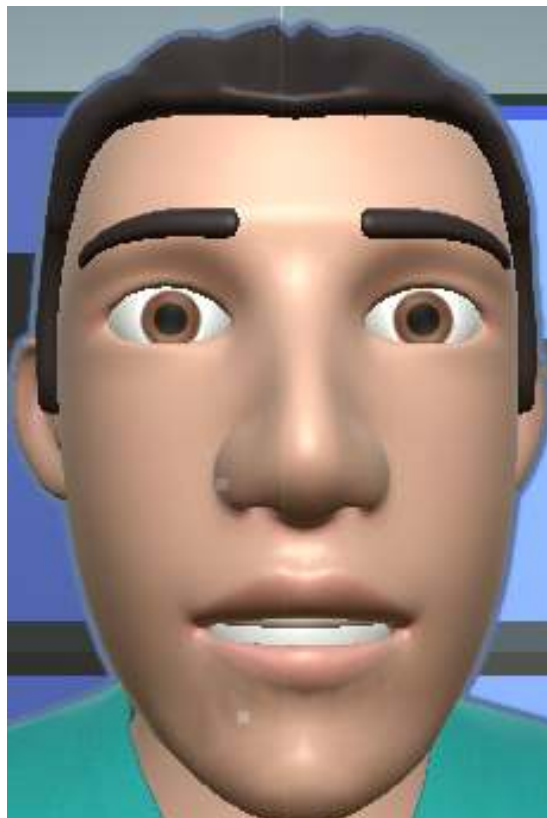


Figure 22: The DD viseme.



Figure 23: The KK viseme.

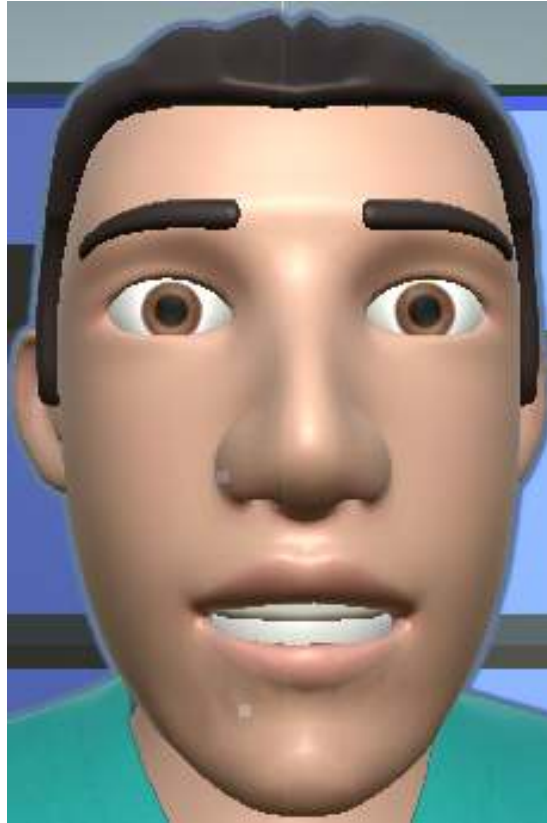


Figure 24: The CH viseme.

The animation can be seen in Figure 24

- **SS**

Corresponding phonemes: s, z

Example: sir, zeal

The animation can be seen in Figure 25

- **NN**

Corresponding phonemes: n, l

Example: lot, not

The animation can be seen in Figure 26

- **RR**

Corresponding phonemes: r

Example: red

The animation can be seen in Figure 27

- **AA**

Corresponding phonemes: a

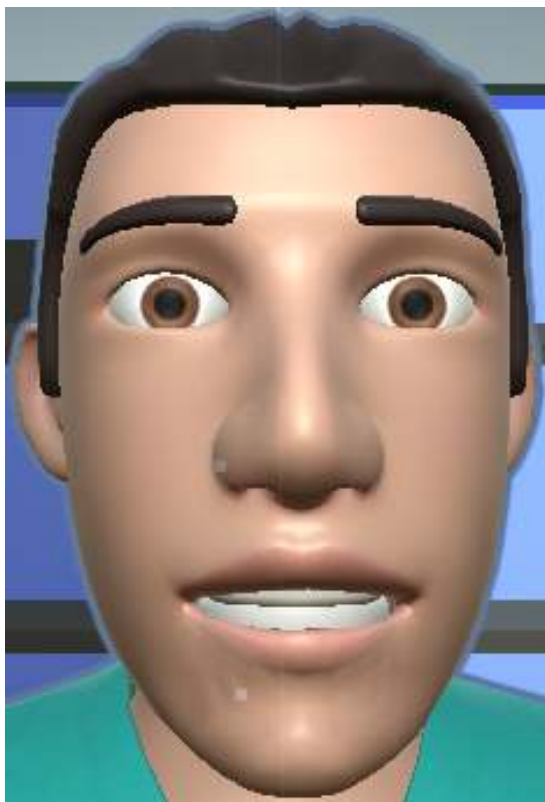


Figure 25: The SS viseme.

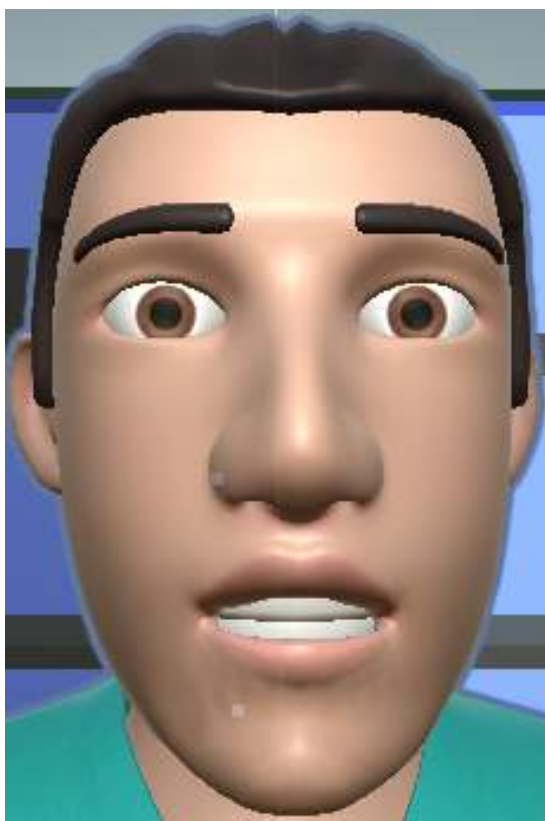


Figure 26: The NN viseme.



Figure 27: The RR viseme.

Example: car

The animation can be seen in Figure 28

- **E**

Corresponding phonemes: e

Example: bed

The animation can be seen in Figure 29

- **I**

Corresponding phonemes: ih

Example: tip

The animation can be seen in Figure 30

- **O**

Corresponding phonemes: oh

Example: toe

The animation can be seen in Figure 31



Figure 28: The AA viseme.



Figure 29: The EE viseme.

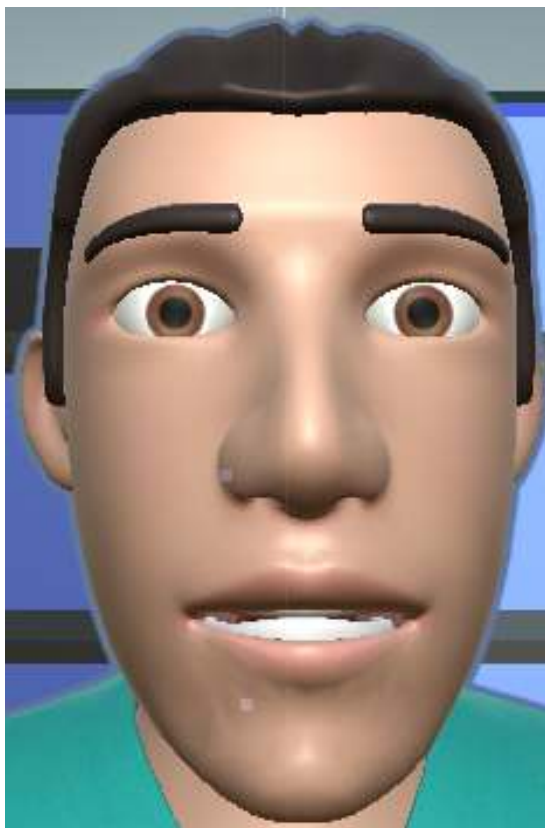


Figure 30: The I viseme.

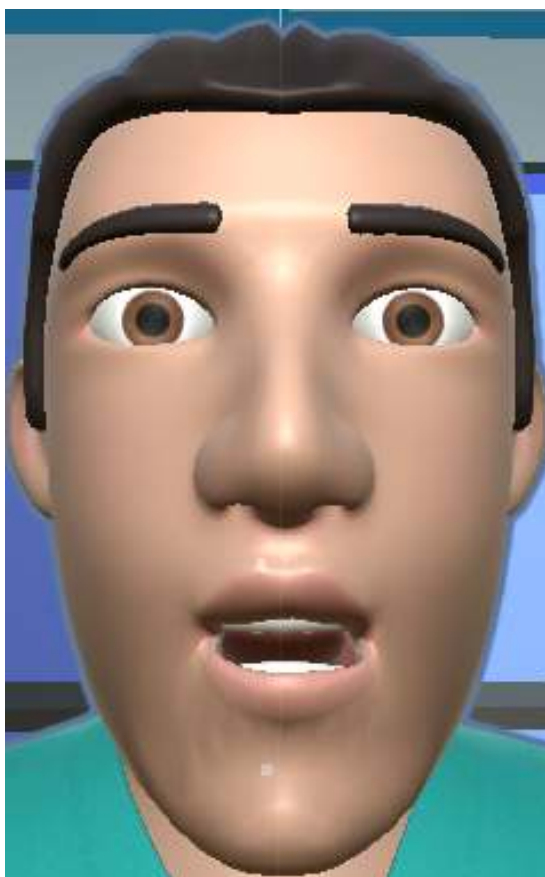


Figure 31: The O viseme.

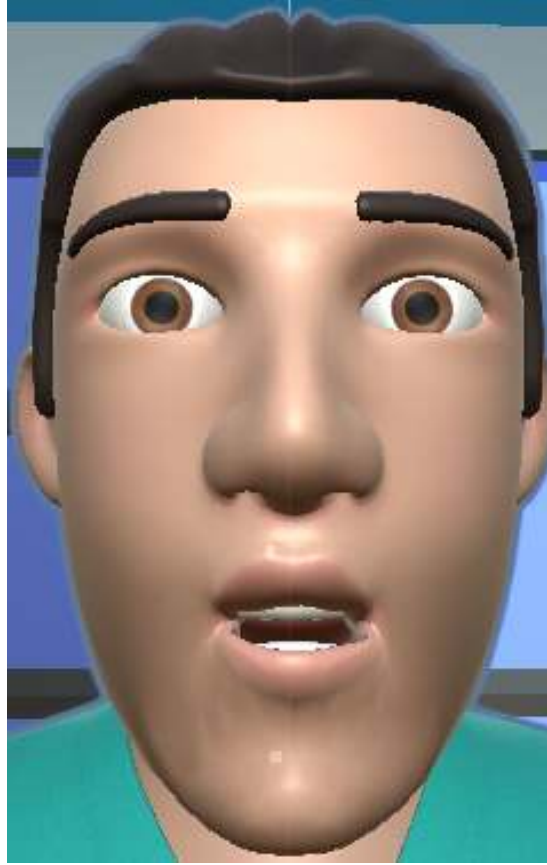


Figure 32: The U viseme.

- U

Corresponding phonemes: ou

Example: book

The animation can be seen in Figure 32

4.3 Animation Controller

According to the Unity Documentation, "An Animator Controller allows you to arrange and maintain a set of Animation Clips and associated Animation Transitions for a character or object. In most cases it is normal to have multiple animations and switch between them when certain game conditions occur". An Animator Controller example can be seen in Figure 33 Unity provides a powerful feature that allows animations to be triggered based on parameters set within the Animator Controller. This feature is highly versatile, empowering users to assign specific parameter values to individual animations. By utilizing the State Machine within the Animator Controller, animations can seamlessly transition between one another. Additionally, users have the flexibility to assign parameters values that fall in between predefined values, facilitating a blending effect between two or

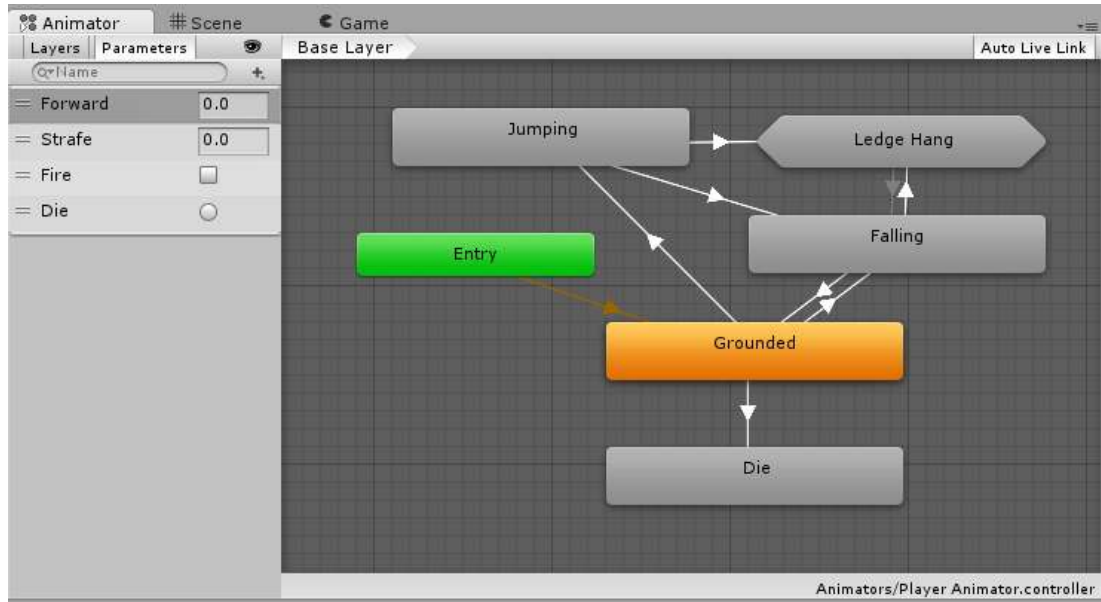


Figure 33: An example of an Animator Controller as depicted in the Unity Documentation.

more animations, with the help of an Animator Controller feature: Blend Trees.

As an example, consider the use of a float parameter named "Speed" within the Animator Controller. If this parameter is assigned a value of 5, and the Animator Controller includes two distinct animations, "Walk" (triggered when the speed is 1) and "Sprint" (triggered when the speed is 10), the model controlled by this Animation Controller will exhibit a combination of characteristics from both "Walk" and "Sprint" animations, thanks to the interpolation capability enabled by the parameter's intermediate value.

4.3.1 Animation Layers

Another feature of the Animation Controller is Animation Layers. The Controller supports one or more layers of animation, which can happen at the same time, each with its own weight upon the model itself. This feature is crucial to this project, as the face of the model executes a lot of animations simultaneously. For each different kind of animation there is a separate Layer, the only exception being the Vowel Layer and the Consonant Layer, which are both about the lip movements of the model and could be implemented as one. I chose to separate them so the structure of the Layers is easier to understand and more readable.

- **Idle Layer**

The only animation in this layer is the idle animation (see Fig. 34), which as mentioned above, it mimics a breathing effect on the model. It is the only layer that its weight is reduced to half, so the animation would be

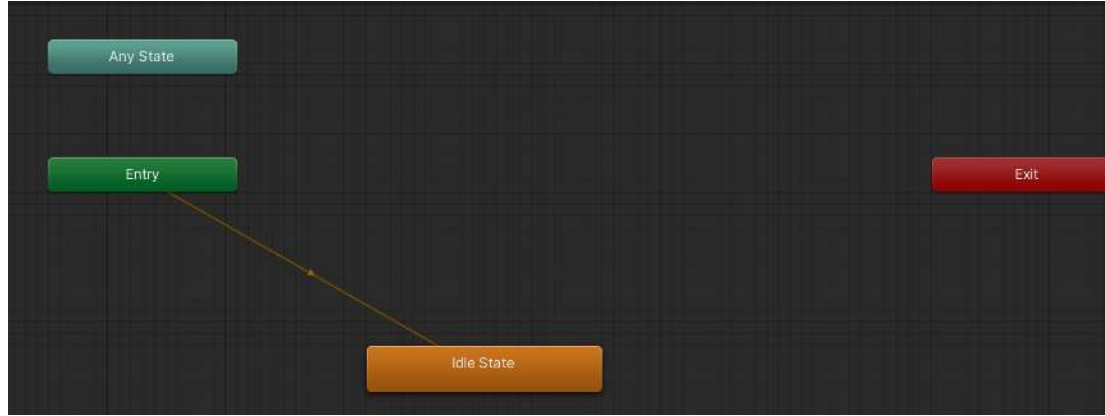


Figure 34: The Idle Animation Layer.

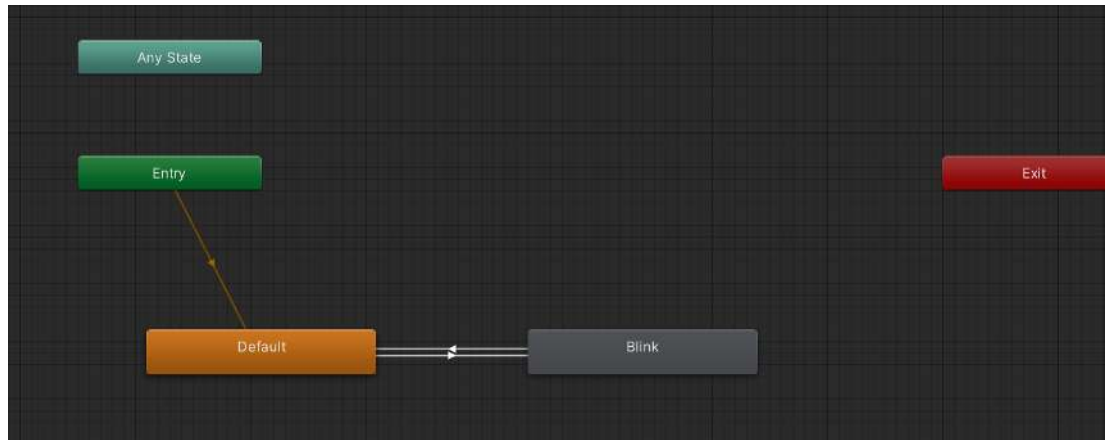


Figure 35: The Blinking Animation Layer.

more subtle, as opposed to others.

- **Blink Layer**

This animation layer (see Figure 35) contains only one animation which revolves around the blinking process of the model, the blinking animation.

- **Vowel Layer**

Although this is called "Vowel Layer" (see Figure 36), it does not contain exactly animations of vowels, but animations of visemes that correspond to vowels. It contains 5 different animations for the visemes AA, E, I, O and U. The animations in this layer focus on the lip movements only.

- **Consonant Layer**

Similarly to the previous one, the Consonant Layer does not contain exactly animations of consonants, but animations of visemes that correspond to consonants (see Figure 37). It contains 10 different animations for the visemes Sil, PP, FF, TH, DD, KK, CH, SS, NN and RR. The animations in this layer focus on the lip movements only.

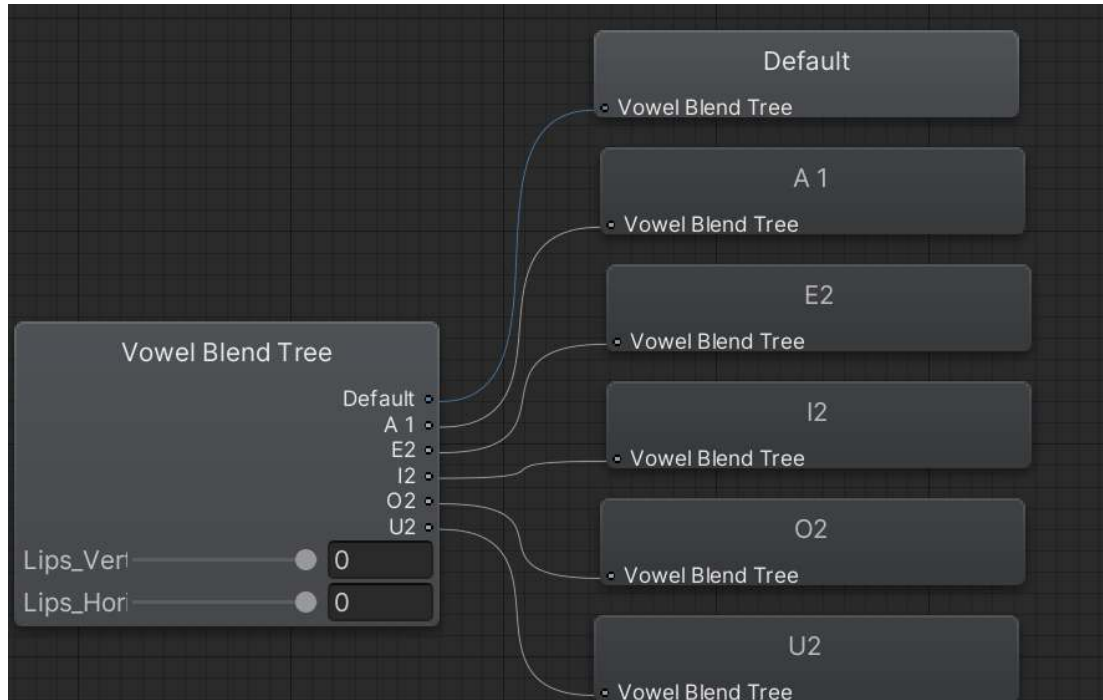


Figure 36: The Vowel Animation Layer.

- **Expression Layer**

In this layer (see Figure 38), the animations include movements of the eyes mostly and, in some cases, minor jaw movements. The jaw movements should not be very impactful on the model itself, as the expression animations can be executed at the same time with the lip movements, where the model also opens and closes its mouth. It contains 6 different animations for the expressions Default, Anger, Disgust, Fear, Happiness, Sadness, Surprise.

5 The User Interface

The User Interface plays a vital role in how users interact with the system. Without a means for users to choose and organize animations over time, the animations themselves cannot run. The UI provides extensive functionality to the project, enabling the manipulation of both existing components and external files, as well as accepting input from a microphone. You can categorize the User Interface into three primary sections: the Input Layers section, the File Input section and the Microphone Input section that can be seen in Figure 39.

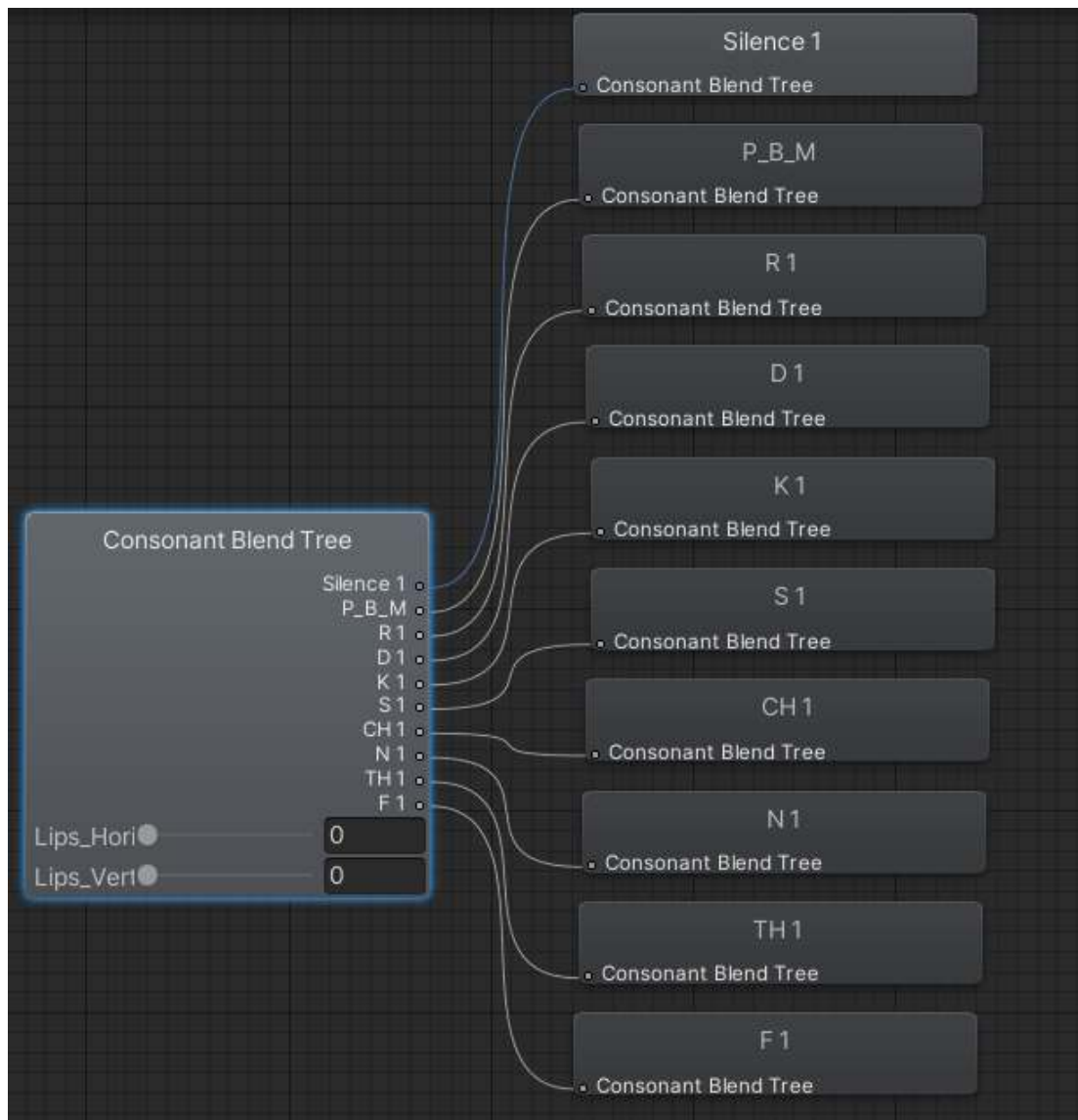


Figure 37: The Consonant Animation Layer.

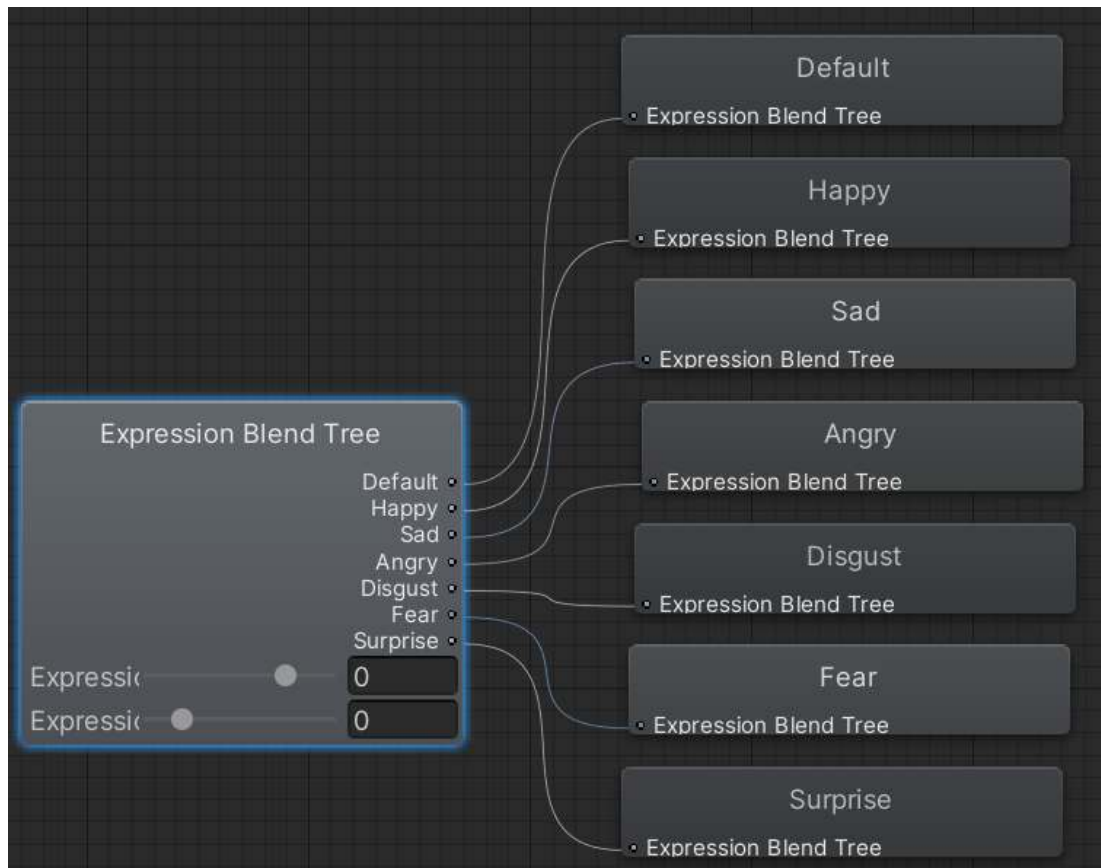


Figure 38: The Expression Animation Layer.



Figure 39: The User Interface, divided in the Input Layer (red box), File Input (green box) and Microphone Input (blue box) sections.

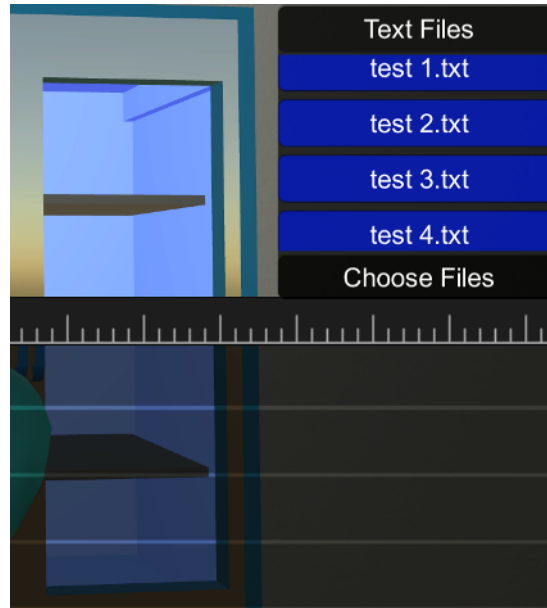


Figure 40: The File Input section, where the user has chosen the Text File category.

5.1 File Input

In this section of the User Interface, the user can choose between different kind of input data that can be dragged and dropped on the Input Layer. The input data can be in either of the four different input formats that were fore-mentioned: Phonemes, Expressions, Text or Audio. Each one has its own signified color (Phoneme is purple, Expression is yellow, Text is blue and Audio is green) so that the user can easier differantiate between them. The Phonemes and the Expressions are already built-in the project, the user cannot add or delete them and are the specific phonemes and expressions we mentioned before. The text and the audio files are loaded from the specific folder that contains them and have the name of the file displayed on them. The user can choose a file category as can be seen in Figure 40 and scroll to see the available choices. If a category is chosen, the other ones disappear until the category button is pressed again.

5.2 Input Layers

In this section of the User Interface, the user can place four different kind of input components from the previous section in their corresponding layer. If the component is dropped onto a layer that does not match its input format, the placement does not occur. If the user successfully inserts a new component, the component latches itself onto the layer. Additionally, specific keys have been bound (Up Arrow and Down Arrow) to scale the layers up or down accordingly. The Layers scale all at the same time and as a result the components they include



Figure 41: The Phoneme layer of the UI, annotated by a red box.

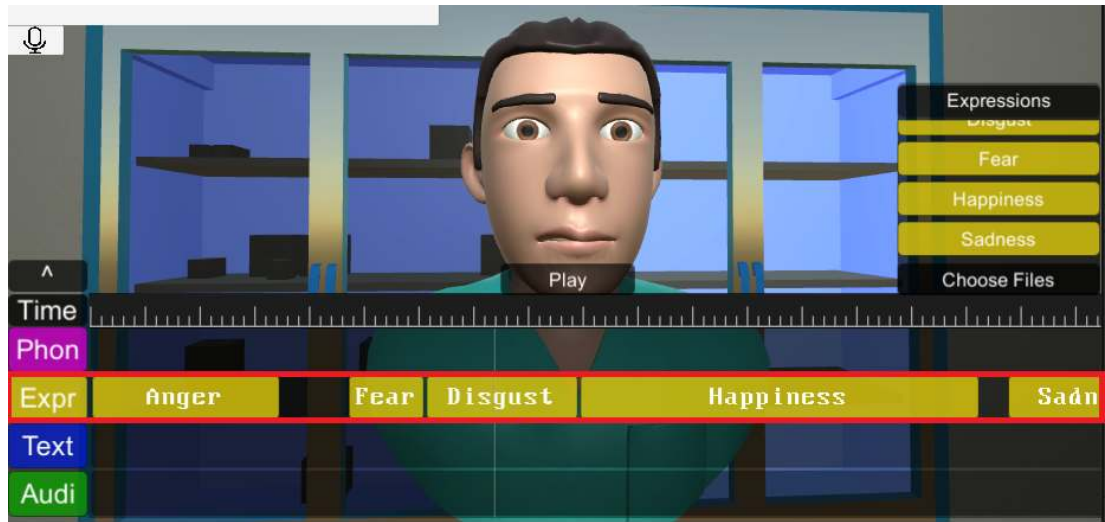


Figure 42: The Expression layer, annotated by a red box.

become larger or smaller.

5.2.1 Phoneme Layer

In this layer the Phoneme components are placed, each one symbolising a specific phoneme which is displayed on top of it (see Figure 41).

5.2.2 Expression Layer

In this layer the Expression components are placed and similarly to the previous layer, each one symbolises a specific expression which is displayed on top of it (see Figure 42).



Figure 43: The Text layer, annotated by a red box.

5.2.3 Text Layer

In this layer the Text components are placed, each one symbolising a specific text file from the DragAndDrop folder. After the component is placed in the Text Layer, it no longer displays the name of the file, but the information (the words) it contains as shown in Figure 43. It is also important to note here that because Phonemes and Text cannot be animated at the same time because they overlap, the system does not allow the user to place one of those 2 at a time window that is already occupied by the a component of the other format. For example if a text file is placed at the beginning and the user tries to place a Phoneme at the same time stamp, the phoneme is automatically deleted, preventing errors.

5.2.4 Audio Layer

In this layer the Audio components are placed, each one symbolising an audio file from the DragAndDrop folder. After the component is placed in the Audio Layer, it erases the file name on top of it and creates a waveform texture of the specific audio it represents. Additionally, the system attempts to create the subtitles of this audio and automatically places a text component at the same time stamp (see Figure 44). This means that the audio component represents the audio of the file alone, the model mimics the subtitles they are generated as the audio is played in the background.

5.3 Components Features

The components placed on the User Interface all have similar attributes, which allow the user to have more freedom manipulating them and producing the wanted



Figure 44: The Audio layer, annotated by a red box, with the corresponding subtitles of the audio files generated (white box).

outcome.

1. Addition

As it was mentioned above, the user can add a new component in the corresponding layer. If the component is added near another, it will stick to the end/beginning of it. Furthermore if it is added in the beginning of the layer itself, it sticks to the beginning as specified by the system.

2. Moving

By dragging a component in the layer after placing, the user can move it in the same layer. While moving a component, the system provides the same features as adding it, where it is possible to stick it to other components or at the beginning.

3. Deletion

By dragging a component outside of its layer, the user has the option to delete this component, in case of an error or in decision of change of the input form.

4. Switching

By dragging a component after placing it on top of another of the same format, it is possible to switch places between the two of them. The system tries to place the one held at the beginning of the other and via versa. In case the components cannot be exactly changed in such a manner because of their size, they are placed stuck to each other.

5. Scaling

If the user drags a component from its edges, the component scales and stretches with the only side moving the one the user holds. For example if the left side is dragged, the component will only extend to the left, keeping its right side the same. When a component is enlarged or shrunk, it takes more or less time accordingly for the model to recreate the outcome. Especially in the case of an audio component, the pitch of the audio is changed so it is clear to the user that the speed of the audio playback is changed.

5.4 Time Layer

Above the Input Layers, there is a Time Layer that visually represents the passage of time from left to right as can be seen in Figure 45. Whenever a user adds a new component to a layer, the component is assigned a starting time based on its position relative to the Time Layer. For instance, if a user adds a text component to the beginning of a layer, the model will begin speaking the text when the user hits the Play Button, at $t=0$ seconds. The content of these components is displayed on the model's face according to their respective start times.

Additionally, there's a needle that is initially positioned at the far left of the input layers. When the user presses the Play Button, this needle starts moving slowly to the right. It serves as a visual representation of the current playback time. As the needle passes over a component, the corresponding animation on the model's face begins. The needle stops moving once all animations have been executed. When the user presses the Play Button again, the needle resets to the beginning of the layers and resumes its rightward movement. The speed at which the needle moves is influenced by the scaling of the layers. If the user has zoomed in on the layers, the needle moves faster because it needs to cover more distance per second due to the enlarged layers.

5.5 Microphone Input

The project also includes support for microphone input. Users can initiate the recording process by pressing a designated button, and once they have finished speaking, they press the button again to stop recording and analyze the input. After the audio has been translated, the text representing the user's speech is displayed on the upper-left part of the screen within a white component. Concurrently, the model attempts to recreate the spoken words through facial animation. However, it's important to note some limitations with this feature. The analysis of the input by the HuggingFace API may not always be accurate, particularly

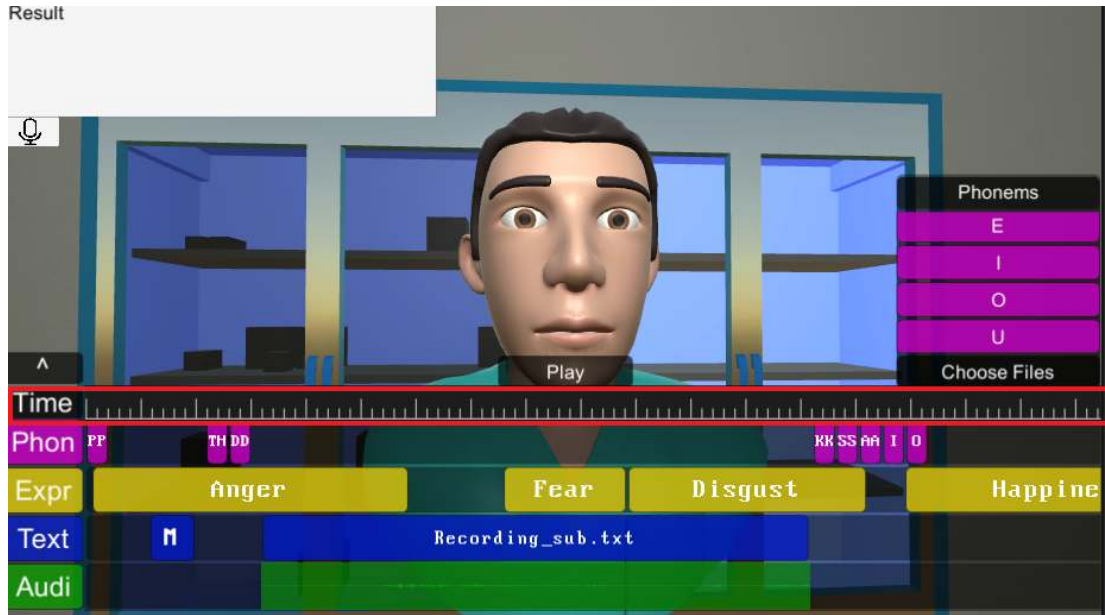


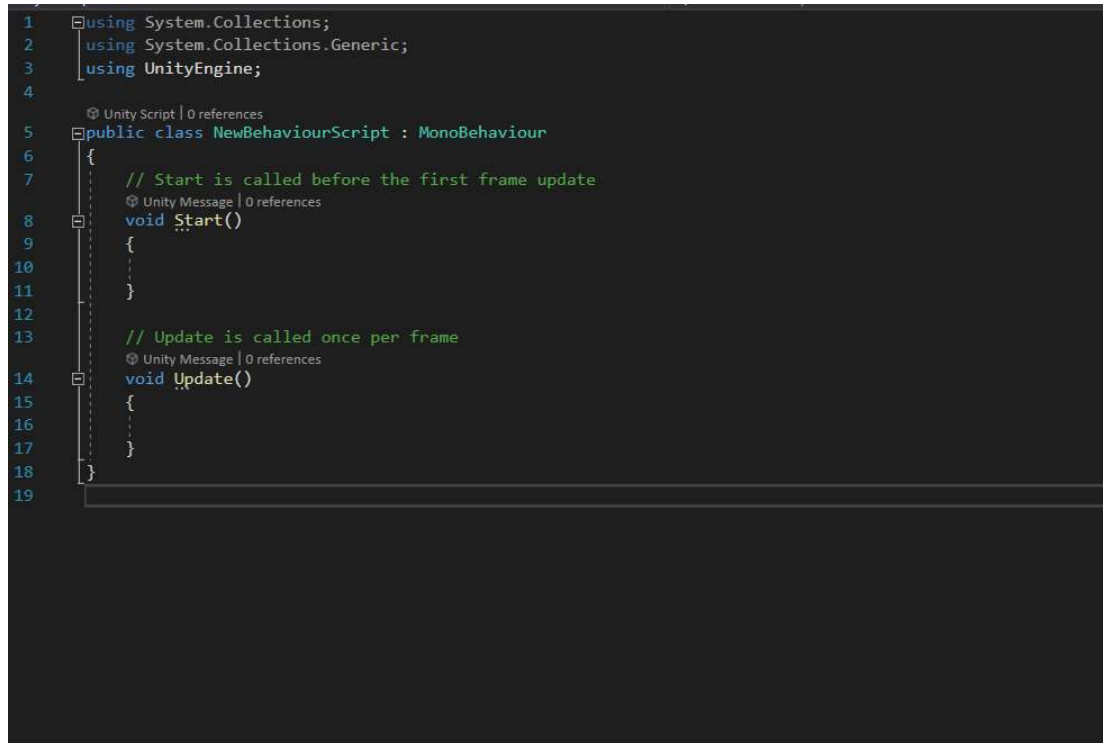
Figure 45: The Time layer, annotated by a red box.

when dealing with microphone input, as sound quality can vary depending on the user’s hardware setup. Additionally, there is room for improvement in terms of real-time transcription. Ideally, the system could transcribe and generate facial expressions in real time as the user speaks, providing a more immediate and synchronized experience with the user’s speech.

6 The Algorithm

It’s worth noting that behind the scenes of this project, there is a crucial algorithm that drives the efficiency of the entire thesis. In Unity, users can create script files in C# to control the behavior of various objects and components. Unity3D has replaced some C# packages with others that are better suited for Unity’s environment. Additionally, many objects in Unity inherit from the MonoBehaviour class, which provides them with valuable functions. For example, the Start() function is executed as soon as the object is created in the scene, while the Update() function is called multiple times per second based on the user’s frame rate (fps). When a new script is made in Unity, the script by default contains those elements as can be seen in Figure 46. To ensure better synchronization and overcome the variable update rate of Update(), FixedUpdate() is employed, which is consistently invoked 50 times per second, regardless of the user’s fps.

In Unity, every object placed on the screen is a GameObject, which can have one or more components that users can add, retrieve, modify, or remove. This feature allows for the creation of events that dynamically alter on-screen objects in response to user actions by manipulating their components through scripting.



```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  [Unity Script | 0 references]
6  public class NewBehaviourScript : MonoBehaviour
7  {
8      // Start is called before the first frame update
9      [Unity Message | 0 references]
10     void Start()
11     {
12     }
13
14     // Update is called once per frame
15     [Unity Message | 0 references]
16     void Update()
17     {
18     }
19 }

```

Figure 46: The Time layer, annotated by a red box.

Notably, every `GameObject` inherits the `MonoBehaviour` class.

The algorithm behind this project involves the interaction of multiple classes, some of which inherit from `MonoBehaviour`, primarily those related to the user interface. Others do not inherit from `MonoBehaviour`. The system operates by scripting the User Interface `GameObject`s to accept user input using Unity3D’s tools and features. This input is then processed by various controllers responsible for handling the facial animations of the model.

6.1 User Interface algorithms

In this section of the algorithm, a great variety of techniques are used to get the user input, process it and achieve an optimal outcome. For starters, as soon as the project starts executing, all the different `GameObject`s that appear on screen and mostly have to do with the User Interface are saved inside a class with a singleton, which is later used to get whatever component is necessary, without having to use costly functions to search for specific components on the Unity3D environment. Afterwards some of the `GameObject`s are disabled or changed so that the User Interface does not appear open right from the beginning and it is left to the user to choose to open the editor. With Event Listeners that are attached to all the different buttons of the UI, the user is able to open and close several parts of the editor and also start playing the animations with the Play Button. The areas that

the user can choose components are displayed according to the button pressed and they are made to be scrollable and expand according to the amount of files inside them. A superclass is created which represents the input file components in general and there are 4 subclasses one for each input format which inherit the previous class. A similar hierarchy exists on the components of the input layers, which are differentiated from the input file ones, as they have different behaviour as they move. All of them implement some interfaces that allow them to be dragged and dropped according to the user's mouse position on screen. The drag and drop feature is also possible with the help of many Utility functions that have been created to ensure the correct placement of the components. Unity3D provides the user with some Components that can be added to a GameObject that can align them in some way, but those Components did not work very well to create the features of the project. That is why a system that arranges the components that were moved around was created from scratch with the help of the Utility functions mentioned above.

6.2 Animation algorithms

As indicated previously, controller scripts were developed to effectively manage the execution of animations in a structured manner. The project adopted a controller-oriented approach to address the control challenges by employing three distinct controllers tailored to different input formats: one for managing expressions, another for audio-related elements, and a third for handling both text and phonemes. The combined controller handles both text and phonemes because they share a similar processing method, given that each text typically comprises numerous phonemes.

Each controller maintains ordered lists that store components and their associated content, when the user drags them into the corresponding layers. These lists are organized based on the elements' start times. When the user initiates playback by pressing the Play Button, each controller populates queues with the elements from their respective lists. These queues contain the actions that need to be executed in sequence.

Utilizing the `FixedUpdate` function, the controllers process the first element in the queue as its start time is reached, and subsequently, the element is removed from the queue. This process continues as long as the queues are not empty. Consequently, the elements added by the user are smoothly processed in synchronization with each other and the movements of the Needle according to the Time Layer.

An additional advantage of this approach, which employs ordered lists to store

elements, is that once the program executes successfully, there is no need to re-add the elements. They are already saved in the lists, allowing the user to simply press the Play Button again to replay the animations.

7 Conclusion

To sum up, this project supports a functional system of processing information given by the user through a highly interactive UI and translating in in the form of movements on the face of the skeleton based model. Although it a complete and functioning project, it still has some room for improvements and new features that can be added to empower it even further. For example the case of animation retargeting could be taken into consideration, as the animations are made for this specific model and the project does not support the transference of the animations on a different skeleton based model. Furthermore there could be some improvements on the microphone feature to achieve real time animations, where the user speaks and the model simultaneously recreates the appropriate facial expressions. All in all, the user has quite a lot of freedom while using this tool thanks to the many features that the User Interface supports and can observe the recreation of his input as realistic facial animations.

References

- [1] Paul Ekman and Wallace V Friesen. *Unmasking the face: A guide to recognizing emotions from facial clues*, volume 10. Ishk, 2003.
- [2] Yingruo Fan, Zhaojiang Lin, Jun Saito, Wenping Wang, and T. Komura. Faceformer: Speech-driven 3d facial animation with transformers. *Computer Vision and Pattern Recognition*, 2021.
- [3] Maja Kocoń. Head movements in the idle loop animation. *IADIS Int. J. Comput. Sci. Inf. Syst.*, 15(2):137–147, 2020.
- [4] Koji Komatsu. Human skin model capable of natural shape variation. *The Visual Computer*, 1988.
- [5] Steve Maddock, James Edge, and Manuel Sanchez. Movement realism in computer facial animation. In *19th British HCI group annual conference, workshop on human-animated characters interaction*, volume 6, pages 1–4, 2005.
- [6] Thalmann Magnenat, Richard Laperrière, and Daniel Thalmann. Joint-dependent local deformations for hand animation and object grasping. Technical report, Canadian Inf. Process. Soc, 1988.
- [7] Tim McLaughlin, Larry Cutler, and David Coleman. Character rigging, deformations, and simulations in film and game production. In *ACM SIGGRAPH 2011 Courses*, pages 1–18. 2011.
- [8] Tomohiko Mukai. Example-based skinning animation. *null*, 2018.
- [9] Verónica Orvalho, Pedro Bastos, Frederic I Parke, Bruno Oliveira, and Xenxo Alvarez. A facial rigging survey. *Eurographics (State of the Art Reports)*, pages 183–204, 2012.
- [10] Dizyi Orlando Putra and Agus Setiawan. The importance of user experience analysis in the design of an education information system application. *Proceedings of the 1st Borobudur International Symposium on Humanities, Economics and Social Sciences (BIS-HESS 2019)*, 2020.
- [11] Samia Dawood Shakir and Ali A Al-Azza. Facial modelling and animation: An overview of the state-of-the art. *Iraqi Journal for Electrical & Electronic Engineering*, 18(1), 2022.

- [12] Kazuki Takashima, Yasuko Omori, Yoshiharu Yoshimoto, Yuichi Itoh, Yoshifumi Kitamura, and Fumio Kishino. Effects of avatar's blinking animation on person impressions. In *Graphics Interface*, pages 169–176, 2008.

A Appendix

A.1 Bone Hierarchy and Bones used

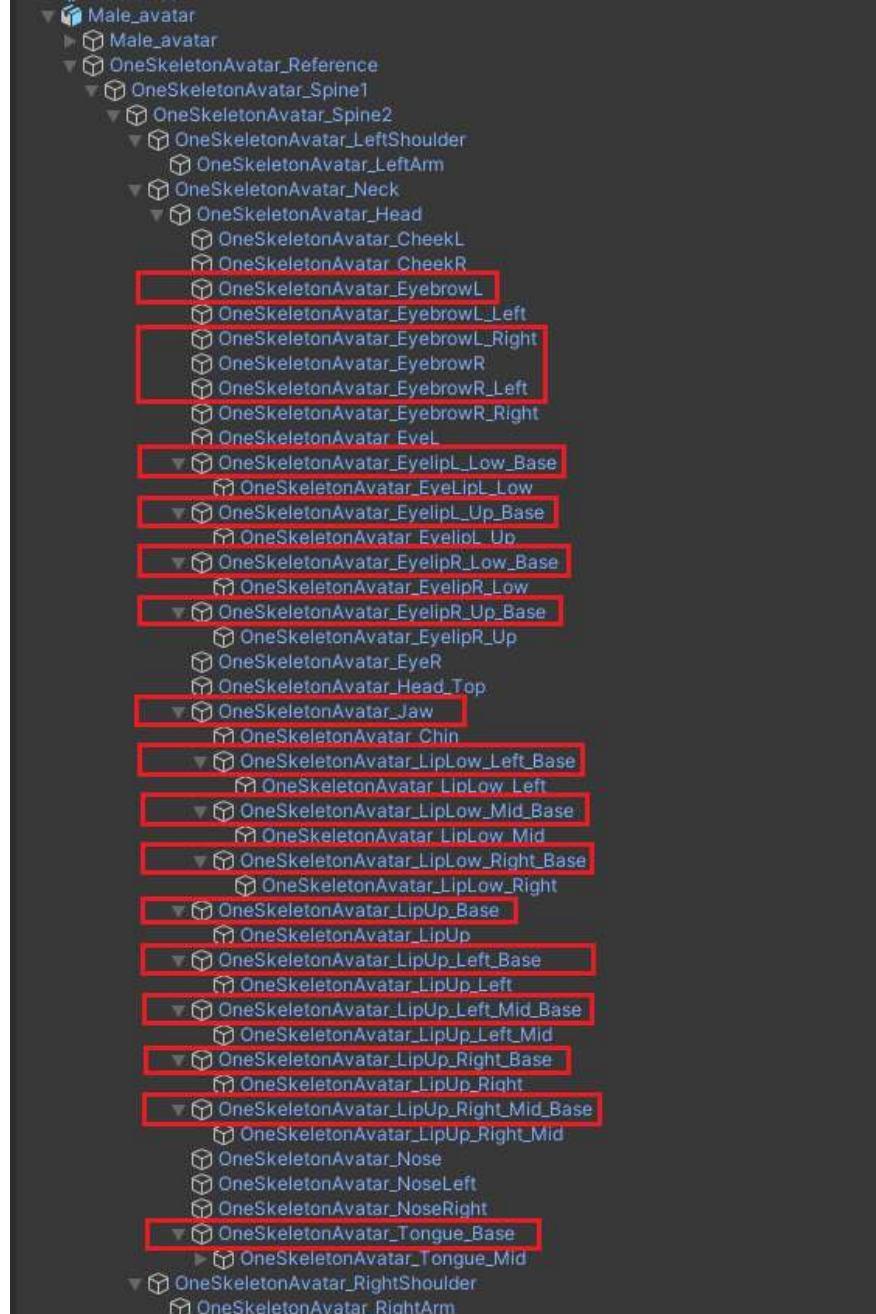


Figure 47: The Bone hierarchy and the bones used, annotated by a red box.

A.2 Controller example

An example of FixedUpdate inside a Controller script (see Fig. 48.)

```

113 0 Unity Messages | 0 references
114 void FixedUpdate()
115 {
116     //If there are available phonemes to animate
117     if (phonemqueue.Count > 0)
118     {
119         //Get the first in line phoneme
120         Phonem nextPhonem = phonemqueue.Peek();
121         if (startTime == -1)
122         {
123             startTime = Time.time;
124             //Get the first phoneme's start time
125             phonemstartTime = nextPhonem.getStartTime();
126             phonemendTime = -1;
127             float ratio = 0.2f / nextPhonem.getDuration();
128             interpolatorDif = ratio * 0.1f;
129             isActive = true;
130         }
131         float actualTime = Time.time - startTime;
132         //If the cooldown time has passed
133         if (actualTime >= phonemstartTime && phonemendTime == -1)
134         {
135             //Delete the phoneme from queue
136             phonemqueue.Dequeue();
137             //Find the right animation to play and change the parameters in the Animator
138             for (int i = 0; i < 15; i++)
139             {
140                 if (phonemes[i] == nextPhonem.GetPhonem())
141                 {
142                     ChangeParameters(i);
143                     break;
144                 }
145             }
146             phonemendTime = nextPhonem.getEndTime();
147             if (phonemqueue.Count == 0)
148             {
149                 startTime = -1;
150             }
151             else
152             {
153                 //Get the next phoneme and its startTime
154                 nextPhonem = phonemqueue.Peek();
155                 float ratio = 0.2f / nextPhonem.getDuration();
156                 interpolatorDif = ratio * 0.1f;
157                 phonemstartTime = nextPhonem.getStartTime();
158             }
159         }
160         if (actualTime > phonemendTime && phonemendTime != -1)
161         {
162             if (phonemqueue.Count != 0)
163             {
164                 //Get the next phoneme and its startTime
165                 nextPhonem = phonemqueue.Peek();
166                 phonemstartTime = nextPhonem.getStartTime();
167                 phonemendTime = -1;
168             }
169             ChangeParameters(0);
170         }
171     }
172     if (phonemqueue.Count == 0 && interpolator == 1)
173     {
174         ChangeParameters(0);
175         phonemstartTime = -1;
176         isActive = false;
177     }
178     // smooth transition from one value to the next
179     xvalue = Mathf.Lerp(xvalue, xnewvalue, interpolator);
180     yvalue = Mathf.Lerp(yvalue, ynewvalue, interpolator);
181     interpolator += interpolatorDif;
182     if (interpolator >= 1) interpolator = 1;
183     // changing the parameters' values to the new ones
184     animator.SetFloat("Lips_Vertical", xvalue);
185     animator.SetFloat("Lips_Horizontal", yvalue);
186 }
187
188
189

```

Figure 48: An example of FixedUpdate inside a Controller script.

A.3 Singleton

A singleton of a public static class is used to get all the component GameObjects we might need (see Figs. 49 and 50).

A.4 Installation Guide

- Install Unity version 2020.3.9f1.
- Download the project via the github link.
- Open the Unity Hub -> Projects -> Open -> Navigate to the folder you downloaded from github.
- Navigate through Assets -> EmptyScene -> Open SampleScene. The Project will now be open and ready to be used.

A.5 How to install & test this framework

- As soon as the user presses the Unity Play Button to start the scene, the User Interface will be mostly empty. To Open the UI the user can press the Arrow Button on the bottom left, which makes the Layers appear.
- The user can import .txt or .wav files inside the Assets/Resources/ DragAndDrop folder, so they can be used later
- The microphone feature can be used by pressing the microphone button on the top left of the screen to start recording from the microphone and after the user gives some speech input, the button must be pressed again so that the speech is analysed, transcribed into text and the animations will start executing on the models face, without pressing the Play Button.
- In case the user wants to use the rest of the features the process consists of the following steps. The user can navigate through the Choose Files Button to the desired input format, choose the component he/she wants to enter and drag and drop it on the corresponding Layer (Note: if the component is not placed correctly on the layer, it will be deleted). The component will be placed with its left side in the same position as the left side of the file just placed.
- If the component is pressed from the sides and dragged, it will start scaling according to the mouse position by having the other side (that was not touched) fixed. If the component is pressed and dragged from elsewhere,

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  Unity Script (1 asset reference) | 90 references
6  public class UIComponents : MonoBehaviour
7  {
8      public static UIComponents singleton;
9
10     public GameObject TimeLayer;
11     public GameObject TextLayer;
12     public GameObject ExpressionLayer;
13     public GameObject AudioLayer;
14     public GameObject PhonemLayer;
15     public GameObject NullLayer;
16
17     public GameObject TextEntry;
18     public GameObject ExpressionEntry;
19     public GameObject AudioEntry;
20     public GameObject PhonemEntry;
21
22     public GameObject TextFilesButton;
23     public GameObject ExpressionFilesButton;
24     public GameObject AudioFilesButton;
25     public GameObject PhonemFilesButton;
26
27     public GameObject TextScrollArea;
28     public GameObject ExpressionScrollArea;
29     public GameObject AudioScrollArea;
30     public GameObject PhonemScrollArea;
31
32     public GameObject Needle;
33
34     public GameObject DragAndDropPanel;
35     public GameObject DragAndDropContainer;
36
37     public GameObject PlayContainer;
38
39     public GameObject SecondCounterPanel;
40
41     public GameObject HRPanel;
42
43     Unity Message | 7 references
44     public void Awake()
45     {
46         if (singleton == null)
47         {
48             singleton = this;
49
50             TimeLayer = GameObject.Find("Time Layer");
51             TextLayer = GameObject.Find("Text Layer");

```

Figure 49: A singleton of a public static class is used to get all the component GameObjects we might need. - Part 1.

```

50 ExpressionLayer = GameObject.Find("Expression Layer");
51 AudioLayer = GameObject.Find("Audio Layer");
52 PhonemLayer = GameObject.Find("Phonem Layer");
53 NullLayer = GameObject.Find("Null Layer");
54
55 TextEntry = GameObject.Find("TextEntry");
56 ExpressionEntry = GameObject.Find("ExpressionEntry");
57 AudioEntry = GameObject.Find("AudioEntry");
58 PhonemEntry = GameObject.Find("PhonemEntry");
59
60 TextFilesButton = GameObject.Find("TextFiles Button");
61 ExpressionFilesButton = GameObject.Find("Expressions Button");
62 AudioFilesButton = GameObject.Find("AudioFiles Button");
63 PhonemFilesButton = GameObject.Find("Phonems Button");
64
65 TextScrollArea = GameObject.Find("TextScrollArea");
66 ExpressionScrollArea = GameObject.Find("ExpressionScrollArea");
67 AudioScrollArea = GameObject.Find("AudioScrollArea");
68 PhonemScrollArea = GameObject.Find("PhonemScrollArea");
69
70 Needle = GameObject.Find("Needle");
71
72 DragAndDropPanel = GameObject.Find("Drag and Drop Panel");
73 DragAndDropContainer = GameObject.Find("Drag and Drop Container");
74
75 PlayContainer = GameObject.Find("Play Container");
76
77 SecondCounterPanel = GameObject.Find("Second Counter Panel");
78
79 HRPanel = GameObject.Find("H/R Panel");
80
81 TextScrollArea.SetActive(false);
82 ExpressionScrollArea.SetActive(false);
83 AudioScrollArea.SetActive(false);
84 PhonemScrollArea.SetActive(false);
85 DragAndDropPanel.SetActive(false);
86 DragAndDropContainer.SetActive(false);
87 PlayContainer.SetActive(false);
88
89 singleton = this;
90 }
91 }
92 }

```

Figure 50: A singleton of a public static class is used to get all the component GameObjects we might need. - Part 2.

it will be moved. If the user moves the component outside of its layer, it is removed, with the exception of moving it far left outside of the layer, where the component sticks to the beginning of the layer. If the component is dropped on empty space inside the layer, it is moved. If it is placed on top of the middle section of another component, it will switch places with it, else if it is placed on the sides of another component, it will stick to the said side.

- The user can place components corresponding to the same time periods, except phonemes and text. In such cases the new component will not be added and be removed instead.
- It is possible to scroll through both the File Input section as well as through the layers.
- By pressing the Up Arrow Key or the Down Arrow Key (The keybinds can change by changing lines 17 and 21 of the Assets/EmptyScene/Scripts/UI/UIControls/ChangeScale.cs script) the user is able to scale up or down the layers themselves
- When the Play Button is pressed, the animations corresponding to the components placed will start playing in the order they are placed relevant to the Time Layer.